

≡ go-ledger

THE LEDGER BOOK

Double-entry accounting with invariants impossible to violate.
Built in the open over twelve weeks.



every debit has an equal credit

Sohag Hasan

2026 · github.com/sohag-pro/go-ledger

A Note Before You Begin	8
How to Read This Book	10

I WEEK 1

11

1 Why I'm Building a Production Go Payment Ledger	12
1.1 Why a ledger, of all things?	12
1.2 Why Go?	14
1.3 The 12-week plan	14
1.4 Week 1: the unglamorous part	15
1.5 My AI companion	16
1.6 Why you should build one too	16
2 Week 1 Interview Q&A	17
2.1 Theme 1: Domain design	17
2.2 Theme 2: Project structure and scaffolding	19
2.3 Theme 3: HTTP server decisions	20
2.4 Theme 4: Tooling and build	21
2.5 Theme 5: Trade-offs and scope	23

II WEEK 2

24

3 Modeling Double-Entry Accounting in Go	25
3.1 The one rule	25
3.2 First decision: how do you even store money?	26
3.3 Modeling the transaction	27
3.4 Proving it holds, hundreds of times	29
3.5 What I deliberately left out	30
3.6 The honest split	30
3.7 Next week	31

4	Week 2 Interview Q&A	32
4.1	Domain design	32
4.2	Data integrity and the invariant	34
4.3	Money representation (ADR-002)	35
4.4	Testing	36
4.5	Trade-offs and defending decisions	37

III WEEK 3

40

5	Postgres Schema for a Multi-Tenant Ledger	41
5.1	The one decision that shapes the rest	41
5.2	Three tables, not five	42
5.3	Multi-tenant from the first migration	43
5.4	The small choices that add up	44
5.5	Proving it works against a real Postgres	44
5.6	The AI companion, week 3	45
5.7	Where this leaves us	45
6	Week 3 Interview Q&A	47
6.1	Schema design	47
6.2	Multi-tenancy and data integrity	49
6.3	Repository and adapter design	50
6.4	Testing	52
6.5	Trade-offs and stack defense	54

IV WEEK 4

55

7	Atomic Transaction Posting in Go	56
7.1	The thing that must never happen	56
7.2	Two ways to be safe	57
7.3	Retrying without making it worse	57

7.4	Belt and suspenders: enforce it in the database too	58
7.5	The afternoon one in six payments failed	59
7.6	What a fresh pair of eyes caught	60
7.7	Where this leaves us	60
8	Week 4 Interview Q&A	62
8.1	Concurrency model	62
8.2	Enforcing the invariant in the database	64
8.3	Architecture and error handling	65
8.4	Observability and operations	67
8.5	Testing	68
8.6	Defending the stack	69

V WEEK 5

70

9	Designing a REST API for a Payment Ledger	71
9.1	Resources and verbs, not remote procedures	71
9.2	Money on the wire	72
9.3	The statement, and the number that is not stored	73
9.4	Paging without lying to the user	73
9.5	When the query generator pushed back	74
9.6	The docs cannot drift	75
9.7	It is real now	75
9.8	The AI companion, week 5	75
9.9	Where this leaves us	76
10	Week 5 Interview Q&A	77
10.1	API design	77
10.2	Account statement	79
10.3	Data and error handling	80
10.4	Server and operations	82
10.5	Defending the stack	83

11	Idempotency Keys and the Immutable Audit Log	86
11.1	The retry problem is not rare, it is the default	86
11.2	The naive version has a hole in it	87
11.3	The part I got wrong on the first pass	88
11.4	The audit log: write it once, never touch it again	89
11.5	Proving it, not hoping it	90
11.6	My AI companion this week	90
11.7	Where this leaves us	91
12	Week 6 Interview Q&A	92
12.1	Idempotency design	92
12.2	Data integrity and the audit log	94
12.3	Concurrency and correctness	96
12.4	API and pagination	97
12.5	Trade-offs	98

13	ADR-001: Why Double-Entry Accounting	101
13.1	Status	101
13.2	Context	101
13.3	Decision	102
13.4	Consequences	102
13.5	Alternatives considered	103
14	ADR-002: Money Representation	104
14.1	Status	104
14.2	Context	104
14.3	Decision	104
14.4	Consequences	105

14.5	Alternatives considered	105
15	ADR-003: Postgres Schema and Repository Layer	106
15.1	Status	106
15.2	Context	106
15.3	Decision	106
15.4	Consequences	108
15.5	Alternatives considered	108
16	ADR-004: Concurrency Control for Transaction Posting	110
16.1	Status	110
16.2	Context	110
16.3	Decision	111
16.4	Observability	112
16.5	Measured baselines	112
16.6	Consequences	113
16.7	Alternatives considered	113
17	ADR-005: Per-Account Currency Integrity	115
17.1	Status	115
17.2	Context	115
17.3	Decision	115
17.4	Consequences	116
17.5	Alternatives considered	116
18	ADR-006: REST API Design	118
18.1	Status	118
18.2	Context	118
18.3	Decision	118
18.4	Consequences	120
18.5	Alternatives considered	121
19	ADR-007: Idempotency Keys and the Immutable Audit Log	122
19.1	Status	122
19.2	Context	122

19.3	Decision	122
19.4	Consequences	125
19.5	Alternatives considered	126
20	ADR-008: Covering Index for Account History Reads	127
20.1	Status	127
20.2	Context	127
20.3	Decision	128
20.4	Consequences	128
20.5	Alternatives considered	129

A Note Before You Begin

A while back a friend paid for dinner before the rest of us could reach for our phones. On the walk home I opened a payment app, sent him my share, and watched my balance drop before I reached the bus stop. Somewhere a system had just moved money between two accounts, atomically, in a way that both of us, the company, and some future auditor could trust completely. I have spent most of my career building on top of systems like that. Using a ledger taught me to operate one. It never taught me to defend one. The gap between those two is this book.

I am Sohag Hasan. For the better part of a decade I have shipped software for fintech and payments teams, most recently as a tech lead, on things like cross-border money transfers and integrations with banks. My day to day has mostly been PHP and Laravel. But the ledger, the part that actually decides whether money is real, was always someone else's library or someone else's table. I wanted to build that part myself, and I wanted to do it in Go. I needed concurrency I could trust: thousands of postings racing on a single account, a race detector that made the failure real, tests that actually caught it. Go fought me on the sharp edges of shared state, which was exactly the lesson I came for.

So I built it in the open, one week at a time. No single week was the hard one. Each had a wall I did not see coming, and understanding arrived slowly, until one day the whole system fit in my head and I could defend any piece of it. That is the bar I was after.

Every week I shipped a piece of the ledger, wrote a post explaining the decisions, and then sat myself down with a list of hard interview questions about what I had just built. I have spent years on both sides of the interview table, and the questions that expose shallow understanding are the same ones I now ask myself. More than once, a question I could not answer sent me straight back to the code. The posts are the story. The questions are the stress test. Reviewing my own work this way changed how I review everyone else's: the bar is no longer "does it pass" but "can you defend it", with the rejected alternatives written down.

I wrote this for the engineer who reaches for a payments API and never thinks about what is underneath, the way I once didn't. If you take one idea from these pages, take this: your balance is not a number someone saved. It is the

sum of every posting that ever touched your account. The history is the truth, and everything else is a cache.

Everything here is real. Every line of code, every schema, every trade-off lives in the open at github.com/sohag-pro/go-ledger, running, tested, and deployed. I have been writing and building in public for years, at notes.sohag.pro, from Dhaka, Bangladesh. This book is part of that habit, not a departure from it. If any of this sounds familiar, you are in the right place.

How to Read This Book

The book is organized the way it was built: one week at a time. Each week has two chapters.

The first chapter of each week is the essay. It tells the story of what I built that week and, more importantly, why: the decisions, the trade-offs, the things I deliberately left out. Read it the way you would read a build log from someone sitting next to you.

The second chapter is the interview. It is a set of questions a senior engineer would face if this project were on their resume, each with the key points a strong answer covers. Use it two ways: to check whether the essay actually landed, and to rehearse defending the design out loud. If you are preparing for backend or systems interviews, this is the part to drill.

At the back you will find the Architecture Decision Records. These are the formal, dated decisions made during the build, kept in the repository and reproduced here. When a chapter says “see ADR-003”, this is where it points. They are terse on purpose.

A few conventions. Code, identifiers, and file paths are set in a monospaced font. Everything in these pages is real and lives in the open at github.com/sohag-pro/go-ledger. You can read the source, run it, or follow the live service at go.sohag.pro.

You can read this front to back, or jump to the week that solves the problem you have in front of you right now. Both work.

WEEK 1

01 Why I'm Building a Production Go Payment Ledger

02 Week 1 Interview Q&A

CHAPTER 1

Why I'm Building a Production Go Payment Ledger

Last night I had dinner with two friends, and one of them paid the whole bill, 1,800 taka, before the rest of us could reach for our phones. So on the way home I opened my payment app, typed 600 taka, and sent him my share. My balance dropped before I reached the bus stop. Then I stopped and thought about what just happened. Somewhere, a system recorded that money left my account and entered his, atomically, in milliseconds, in a way that both of us, the payment company, and possibly an auditor three years from now can trust completely.

That system is called a ledger, and I'm going to build one. In public, open source, over twelve weeks, with a blog post every week. This is post 1 of 12.

1.1 Why a ledger, of all things?

Most side projects die for one of two reasons: they're too ambitious to finish, or too trivial to matter. A todo app teaches you nothing new. A "build your own database" project takes two years and ships never.

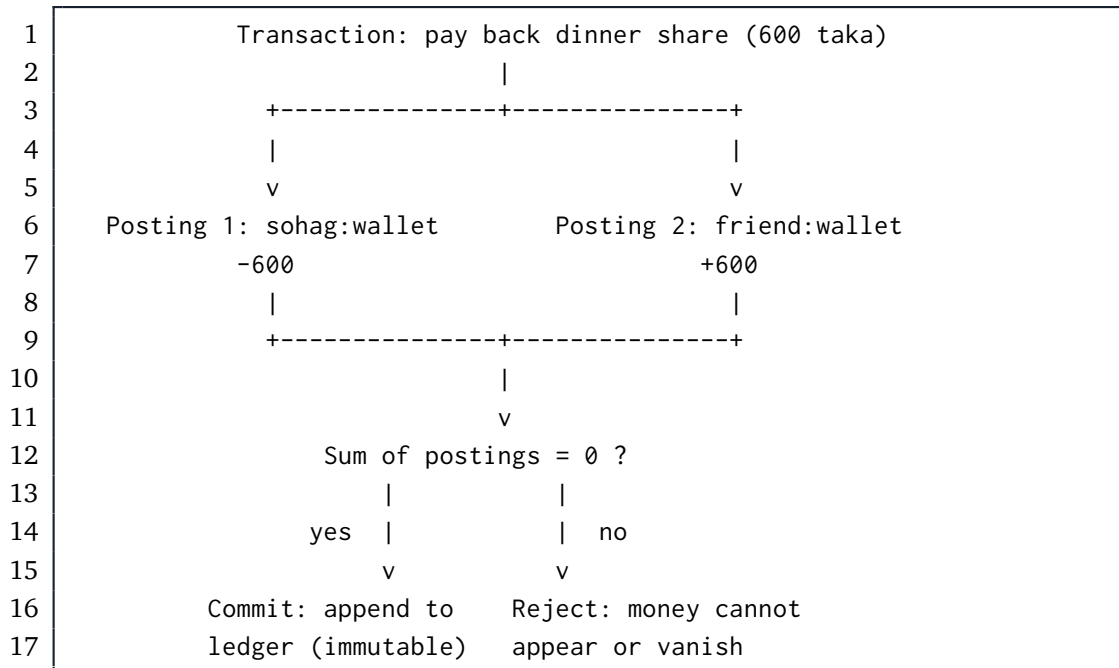
A payment ledger sits in the sweet spot. It's small enough to build in twelve weeks, and deep enough that doing it *right* forces you through almost everything that makes backend engineering hard: data integrity, concurrency, idempotency, observability, and deployment.

Every fintech company has a ledger at its core, and most are built on an idea older than the printing press: **double-entry bookkeeping**. The rule is simple. Money never appears or disappears. It only *moves*. Every transaction is recorded as two or more postings that must sum to zero. Your balance isn't a number someone updated; it's the sum of every posting that ever touched your account. The history *is* the state.

Back to my dinner. Here's what paying my friend back his 600 taka looks like as a double-entry transaction:

Posting	Account	Amount
1	sohag:wallet	-600
2	friend:wallet	+600
Sum		0

Two postings, summing to zero. If a bug ever produces a transaction where the sum isn't zero, money was created or destroyed out of thin air, and the system must reject it before it persists:



That one constraint is why this project is interesting to build:

- **Correctness is non-negotiable.** A bug in a CRUD app shows a stale page. A bug in a ledger creates or destroys money. The invariant must hold under any failure, any concurrency, any retry.
- **The hard problems are real ones.** What happens when two requests post to the same account simultaneously? When a client retries a payment because the network dropped the response? When the database dies mid-transaction? These aren't interview puzzles; they're Tuesday.
- **It's bounded.** The domain fits in your head. Accounts, transactions, postings, balances. No ML, no infinite feature surface. The depth is vertical, not horizontal.

1.2 Why Go?

I want a language where the production story is boring. Go gives me a single static binary, a stellar standard library (`net/http`, `log/slog`, `database/sql` and `friends`), first-class concurrency for the stress tests I have planned, and an ecosystem where the “right” choice for most infrastructure questions is well-trodden.

The stack is locked upfront, because relitigating decisions mid-project is how twelve weeks becomes thirty:

- **Router:** `chi` (idiomatic, minimal)
- **Database:** Postgres with `pgx` + `sqlc` for type-safe queries, `goose` for migrations
- **API:** REST first, gRPC later (with `buf` for proto management)
- **Testing:** standard library + `testcontainers-go`, `k6` for load, property-based tests for the balance invariant
- **Observability:** OpenTelemetry traces, `slog` structured logs, Prometheus metrics
- **Deployment:** Docker image, deployed to a VPS via GitHub Actions CI/CD

No Kubernetes, no Kafka, no event sourcing, no multi-currency. Not because they're bad, but because scope discipline is the difference between shipping and abandoning. Single currency, one region, no admin UI. Everything cut from v1 becomes a follow-up post if the project earns it.

1.3 The 12-week plan

1. **Scaffolding:** repo, module layout, tooling, hello ledger (*this post*)
2. **Domain model:** double-entry accounting in Go types; invariants the compiler helps enforce
3. **Postgres schema + repository layer:** append-only postings, integration tests against real Postgres
4. **Transaction posting service:** atomicity, `SERIALIZABLE` isolation, correctness under 10,000 concurrent posts
5. **REST API:** `chi`, validation, RFC 7807 errors, OpenAPI
6. **Idempotency keys + immutable audit log:** the two patterns that make fintech APIs boring (in a good way)
7. **gRPC layer:** shared domain services behind both protocols

8. **OpenTelemetry**: one trace from HTTP handler to SQL query
9. **Testing**: unit, integration, property-based, load, and a little chaos
10. **Docker + infrastructure**: multi-stage builds, distroless, tiny images
11. **Live deployment + CI/CD**: push to main, live in minutes
12. **Launch + retrospective**

1.4 Week 1: the unglamorous part

This week was scaffolding, deliberately light. The goal: anyone can clone the repo, run `make run`, and hit a health check.

The layout follows the standard Go convention:

```
1 go-ledger/├─
2   cmd/server/      # main.go, wiring only, no logic├─
3   internal/        # domain code lives here, unimportable from outside
4   │               └─
5   pkg/             # public packages, if any earn the spot├─
6   docs/adr/        # architecture decision records├─
7   Makefile         # run, build, test, lint, dev├─
8   Dockerfile       # multi-stage skeleton, distroless base├─
9   .golangci.yml    # lint config├─
10  .air.toml         # hot reload
```

The server itself is about seventy lines of standard library: an `http.ServeMux` (the 1.22+ pattern-matching routes mean I don't even need `chi` yet), `slog` JSON logging from day one, and graceful shutdown on `SIGTERM`, because writing it now is free and bolting it on later never happens.

```
1 $ curl localhost:8080/healthz
2 {"status": "ok"}
```

Not impressive. But it builds clean, lints clean (`golangci-lint` with `gosec` and friends enabled from commit one), and ships with the project's first architecture decision record: **ADR-001, why double-entry**. Writing ADRs for a solo project feels ceremonial until you're in week 9 trying to remember why you rejected event sourcing. Future me will thank present me.

1.5 My AI companion

Full transparency: I'm not building this alone. I have an AI pair, **Claude**, running in my terminal via Claude Code.

This week it handled the unglamorous parts: installed the Go toolchain, scaffolded the project layout, wrote the Makefile and lint config, drafted the first ADR, and caught its own lint failures before I saw them. That's the deal I've settled on: Claude accelerates the mechanical work, and I own every decision that matters. The architecture, the invariants, the trade-offs in the ADRs, and the understanding stay mine. A ledger where I can't explain *why* every line exists would defeat the entire point of this project.

I'll be honest about the split as the series goes. When Claude writes something substantial, I'll say so. When I overrule it, that's probably worth a paragraph too; those disagreements tend to be where the interesting engineering lives. If you're curious how an AI-assisted workflow holds up across a twelve-week production build, that's now a quiet subplot of this series.

1.6 Why you should build one too

Because "I read about idempotency keys" and "I watched 100 goroutines try to corrupt my ledger and fail" are different kinds of knowledge. The second kind is what production engineering actually is, and you can't get it from blog posts, including this one. You get it by building the thing, breaking it, and fixing it.

The repo is open source at github.com/sohag-pro/go-ledger. Follow along, steal the plan, or build your own. Next week: modeling double-entry accounting in Go's type system, and making the balance invariant impossible to violate.

CHAPTER 2

Week 1 Interview Q&A

Practice set for defending go-ledger in a senior engineering interview. Everything here is answerable from this week's actual deliverables: `cmd/server/main.go`, the project layout, the Makefile, the Dockerfile, `.golangci.yml`, and `docs/adr/001-why-double-entry.md`.

2.1 Theme 1: Domain design

2.1.1 1. Warm-up: Walk me through why you chose double-entry accounting for this ledger instead of a simple balances table.

What a strong answer covers:

- A mutable balance row tells you what the balance is, never how it got there; the audit trail and the state drift apart.
- Double-entry makes every transaction a set of postings that must sum to zero, so money structurally cannot appear or vanish.
- Balances become derived values (sum of postings), so the history is the source of truth.
- ADR-001 records this reasoning, including the rejected alternatives.

2.1.2 2. Your ADR says postings are append-only and balances are never stored as mutable primary state. What does that buy you, concretely?

What a strong answer covers:

- Append-only writes sidestep read-modify-write races on a balance row entirely.

- Every balance is explainable: it is an aggregation over immutable facts.
- A corrupted cache can be rebuilt from postings; the reverse is not true.
- The trade-off is that reads become aggregations, accepted in ADR-001 with a rebuildable cache as the escape hatch.

2.1.3 3. Challenge a locked decision: Double-entry is centuries old and adds friction. Clients now have to submit balanced postings instead of “add 600 to account X”. Why is that complexity worth it for a v1?

What a strong answer covers:

- The friction is the feature: an unbalanced request is a bug surfaced at the API boundary instead of corrupted money discovered months later.
- “Add 600 to X” has no answer to “from where?”, which is exactly the question auditors and reconciliation ask.
- The domain is fintech; correctness is the product. A todo app can take the simpler model.
- The model matches what payment partners and accountants already expect, reducing integration friction later.

2.1.4 4. Hard follow-up: ADR-001 claims append-only postings reduce concurrency control to “serializing transaction inserts”. Two requests race to post against the same account at the same moment. What can still go wrong, and what is your plan?

What a strong answer covers:

- Inserts of postings themselves do not conflict, but any read-then-decide logic does (e.g. “reject if balance would go negative” reads a balance that another in-flight transaction is changing).
- This is a write skew shaped problem; append-only does not make it disappear.
- The plan (Week 4) is a DB transaction at SERIALIZABLE isolation plus row locks on involved accounts, with the invariant also enforced by a Postgres CHECK constraint as a backstop.

- Knowing the failure mode before writing the code is why the ADR and plan exist.

2.2 Theme 2: Project structure and scaffolding

2.2.1 5. Warm-up: Explain your repo layout. Why `cmd/server/`, `internal/`, and `pkg/`?

What a strong answer covers:

- `cmd/server/main.go` is wiring only: config, server start, shutdown. No business logic, so the entry point stays trivially readable.
- `internal/` is compiler-enforced privacy; no external module can import the domain code, which keeps the public API surface deliberate.
- `pkg/` exists but is empty; packages must earn public status, not default to it.
- This mirrors the standard Go project convention, so any Go developer can navigate it immediately.

2.2.2 6. Why did you write an ADR for a solo side project? Who is it for?

What a strong answer covers:

- Future self in week 9, deciding whether to relitigate event sourcing, reads why it was rejected in week 1.
- Interviews and open source contributors get the reasoning, not just the result.
- ADRs force the alternatives to be written down at decision time, when the trade-offs are freshest.
- Cheap to write now, expensive to reconstruct later.

2.2.3 7. You wrote graceful shutdown handling in `main.go` in week 1, before there is any real traffic. Premature?

What a strong answer covers:

- The shutdown path (signal.NotifyContext on SIGINT/SIGTERM, srv.Shutdown with a 10s timeout) is about 15 lines now and free to write.
- Retrofitting it later competes with feature work and never happens; meanwhile every deploy hard-kills in-flight requests.
- For a ledger specifically, killing a request mid-transaction is the exact class of bug the project exists to prevent.
- It also makes the service well-behaved under Docker and CI from the first deploy, since both deliver SIGTERM.

2.3 Theme 3: HTTP server decisions

2.3.1 8. The stack locks in chi as the router, but main.go uses the standard library http.ServeMux. Contradiction?

What a strong answer covers:

- Go 1.22+ ServeMux supports method-prefixed patterns (GET /healthz), enough for two routes.
- chi earns its place when the middleware chain arrives (logging, recovery, request ID) in Week 5.
- Adding a dependency before its value exists is scope creep at the dependency level.
- The handlers will not change when chi lands; only the wiring in main.go does.

2.3.2 9. Challenge a locked decision: Why chi at all then? The 1.22 ServeMux handles routing, and you clearly know it. Defend the dependency.

What a strong answer covers:

- chi adds composable middleware, route groups, and URL param handling that ServeMux still lacks, all on stdlib types (http.Handler everywhere).
- Because chi is stdlib-compatible, the exit cost is near zero; handlers never know chi exists.

- The alternative is hand-rolling middleware chaining and mounting, which is rebuilding chi badly.
- Locked decisions exist to prevent mid-project relitigating; the ADR-style reasoning happened once, upfront.

2.3.3 10. Why does your `http.Server` set `ReadHeaderTimeout`, and what would happen without it?

What a strong answer covers:

- Without it, a client can open a connection and trickle header bytes forever (slowloris), pinning server resources.
- gosec flags the omission; the lint config enforces it from commit one.
- 5 seconds bounds the attack window without affecting legitimate clients.
- Related timeouts (read, write, idle) will matter once real handlers exist; this is the minimum safe default for a health-check server.

2.3.4 11. Hard follow-up: Your server goroutine sends errors into a channel, and main selects on that channel and a signal context. Why this shape instead of just calling `ListenAndServe` and handling the error?

What a strong answer covers:

- `ListenAndServe` blocks, so it must run in a goroutine if main also waits on signals.
- The buffered error channel (size 1) lets the goroutine exit even if main has already moved on, avoiding a goroutine leak.
- The select races startup failure (port in use) against shutdown signal; both paths produce a clean exit with the error logged.
- `http.ErrServerClosed` is filtered because it is the expected result of `Shutdown`, not a failure.

2.4 Theme 4: Tooling and build

2.4.1 12. Your Makefile builds with `CGO_ENABLED=0` and tests with `-race`. Those flags look contradictory. Explain.

What a strong answer covers:

- `CGO_ENABLED=0` for builds produces a fully static binary, which is what lets the Dockerfile use a distroless static base image.
- The race detector requires `cgo`, so `make test` simply does not set the variable; build and test have different goals.
- Race detection from day one is cheap insurance for a project whose week 4 milestone is 10,000 concurrent posts.
- Knowing why each flag exists beats cargo-culting a Makefile.

2.4.2 13. Why distroless for the final Docker image instead of alpine or scratch?

What a strong answer covers:

- Distroless static ships CA certificates, `tzdata`, and a nonroot user, which scratch lacks and the app will need (TLS to Postgres, HTTPS egress).
- No shell and no package manager shrinks the attack surface compared to alpine.
- The nonroot tag means the container does not run as root by default.
- Image size target (<20MB) supports fast pulls and deploys in the Week 11 CI/CD pipeline.

2.4.3 14. Your lint config enables `gofumpt`, `gosec`, `revive`, and friends from the first commit. Why front-load lint strictness instead of adding it when the codebase grows?

What a strong answer covers:

- Lint debt compounds; enabling `gosec` on 10,000 existing lines produces a wall of findings nobody fixes.
- It already paid off in week 1: `errcheck` caught unchecked `Fprint` returns and `revive` demanded the package comment.

- Style arguments (gofumpt) are settled by the tool, not by future code review back-and-forth.
- CI (Week 9) will reuse the exact same config, so local clean means CI clean.

2.5 Theme 5: Trade-offs and scope

2.5.1 15. Hard follow-up: Your v1 cuts multi-currency, and ADR-001 punts money representation to ADR-002. An interviewer pushes: “You are building a ledger and have not decided how to represent money. Is that not the first decision?”

What a strong answer covers:

- The balance invariant (postings sum to zero) is representation-independent; it holds for int64 cents and for decimals alike, so the domain model is not blocked.
- Deferring with a written deadline (ADR-002, Week 2) is different from not deciding; the decision lands before the first `Money` type is written.
- Single currency in v1 removes the hardest representation pressure (FX, mixed-currency transactions) deliberately, recorded in the scope discipline list.
- Sequencing decisions to land exactly when code needs them is the discipline that keeps a 12-week plan at 12 weeks.

WEEK 2

01 Modeling Double-Entry Accounting in Go

02 Week 2 Interview Q&A

CHAPTER 3

Modeling Double-Entry Accounting in Go

A few years ago I shipped a bug that moved money and forgot to take it from anywhere. The code added a credit to one account and, because of a botched early return, never wrote the matching debit. Nothing crashed. No test failed. The balance just quietly grew, like a bathtub with the drain unplugged, until someone in finance noticed the numbers didn't tie out and the hunt began.

That bug is the reason this week exists. Last week I explained *why* I'm building a payment ledger and got a "hello ledger" health check responding. This week is the part that would have caught that bug at compile time, or close to it: the domain model. No database yet, no HTTP. Just Go types and the one rule they exist to protect. This is post 2 of 12.

3.1 The one rule

Double-entry accounting has a single invariant that everything else hangs off:

Every transaction is two or more postings, and those postings must sum to zero.

Money never appears or vanishes. It moves. If I pay a friend back 600 taka, one account goes down by 600 and another goes up by 600, and the two cancel out. The sum is zero. If the sum is ever anything but zero, money was created or destroyed, and that transaction is a bug wearing a transaction's clothes.

My whole goal for the week: make a transaction that doesn't balance hard to even build, and impossible to validate. If the type system and one small method stand guard, the bathtub bug from my past can't happen here.

3.2 First decision: how do you even store money?

Before modeling a transaction I had to answer a deceptively boring question: what is an amount? This got its own architecture decision record (ADR-002), because getting it wrong is expensive and quiet.

The wrong answer is `float64`. Floating point can't represent most decimal fractions exactly. In Go, `0.1 + 0.2` is `0.30000000000000004`. That rounding error is invisible on one transaction and catastrophic across a million of them. Money and floats do not mix.

The answer I picked: store every amount as a signed `int64` count of the currency's smallest unit. For US dollars that's cents. So \$10.50 is the integer `1050`. Exact, fast, no allocation, and it maps straight onto a Postgres `BIGINT` when the database arrives next week. The tradeoff is that I have to guard against overflow myself, which turns out to be a few lines.

Here's the type. The fields are unexported so a `Money` can only be built through a constructor that validates the currency:

```
1 type Money struct {
2     amount    int64    // signed minor units: cents for USD
3     currency  Currency
4 }
5
6 func NewMoney(amount int64, currency Currency) (Money, error) {
7     if err := currency.Validate(); err != nil {
8         return Money{}, err
9     }
10    return Money{amount: amount, currency: currency}, nil
11 }
```

The interesting part is arithmetic. Adding two amounts can overflow `int64`, and an overflow that silently wraps a balance from a huge positive to a huge negative is exactly the kind of horror this project is meant to avoid. So `Add` checks and returns an error instead of wrapping:

```
1 func (m Money) Add(other Money) (Money, error) {
2     if m.currency != other.currency {
3         return Money{}, ErrCurrencyMismatch
4     }
5     sum := m.amount + other.amount
6     // Overflow happened if both operands share a sign but the result
7     // flipped.
8     if (m.amount > 0 && other.amount > 0 && sum < 0) ||
```

```

8         (m.amount < 0 && other.amount < 0 && sum > 0) {
9             return Money{}, ErrOverflow
10        }
11        return Money{amount: sum, currency: m.currency}, nil
12    }

```

Two things fall out of this for free. Different currencies can't be added, so a multi-currency mistake (a thing explicitly out of scope for v1) can't silently corrupt a sum. And overflow is a typed error you can test, not undefined behavior you discover in production.

3.3 Modeling the transaction

A **posting** is a single signed entry against one account. The sign carries the direction: a positive amount is a debit, a negative amount is a credit. One field, not two, because the sign already tells you everything.

```

1 type Posting struct {
2     AccountID string
3     Amount    Money
4 }
5
6 type Transaction struct {
7     ID        string
8     Postings []Posting
9 }

```

Back to paying my friend 600 taka. As postings:

Posting	Account	Amount
1	sohag:wallet	-600
2	friend:wallet	+600
Sum		0

Now the chokepoint. `Transaction.Validate()` is the single place in the domain where the invariant lives. It needs at least two postings, every posting has to name an account, every leg has to be the same currency, and the signed amounts have to sum to exactly zero:

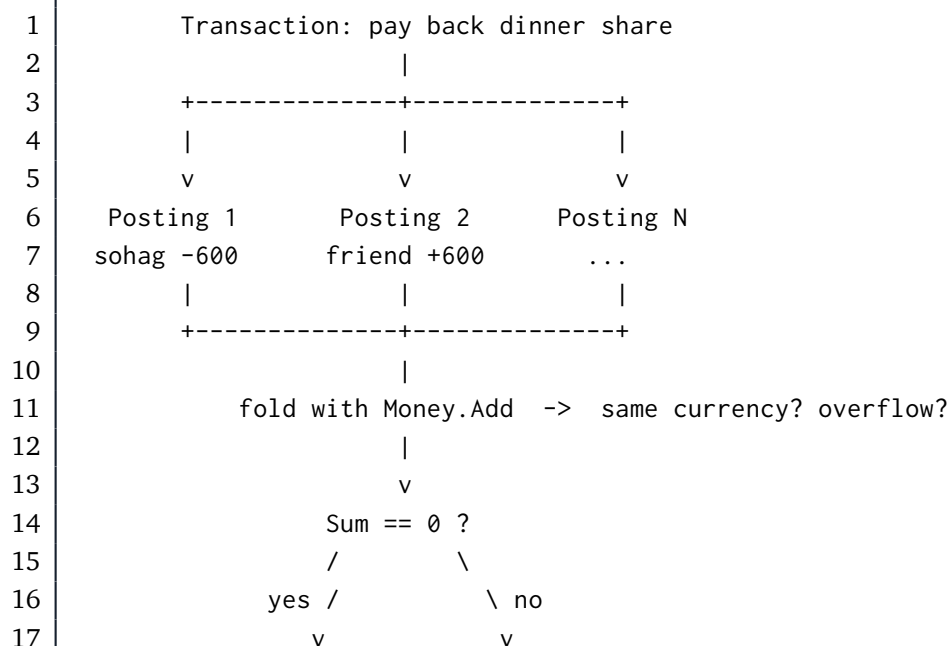
```

1 func (t Transaction) Validate() error {
2     if len(t.Postings) < 2 {
3         return ErrTooFewPostings
4     }
5     for _, p := range t.Postings {
6         if err := p.Validate(); err != nil {
7             return err
8         }
9     }
10    sum := t.Postings[0].Amount
11    for _, p := range t.Postings[1:] {
12        next, err := sum.Add(p.Amount) // surfaces currency mismatch and
13                                       // overflow
14        if err != nil {
15            return err
16        }
17        sum = next
18    }
19    if !sum.IsZero() {
20        return ErrUnbalanced
21    }
22    return nil
23 }

```

Notice how the currency check and the overflow check come for free. I don't write them in `Validate` at all. They happen because the running sum is built with `Money.Add`, and `Add` already refuses to mix currencies or wrap. The small type did the work so the big method didn't have to.

Here's the same flow as a picture:



```

18     Validate passes    ErrUnbalanced
19     (safe to persist) (rejected)

```

Every arrow that could go wrong has a named error waiting for it: `ErrTooFewPostings`, `ErrInvalidPosting`, `ErrCurrencyMismatch`, `ErrOverflow`, `ErrUnbalanced`. When something fails, the caller knows exactly which rule it broke.

3.4 Proving it holds, hundreds of times

Table-driven unit tests cover the cases I can think of: a balanced two-leg transaction, a balanced three-leg one, an unbalanced one, a single posting, a currency mismatch, an empty account ID. Good, but those are the cases *I* imagined. The bug from my past was one I hadn't imagined. So the balance invariant gets a second kind of test.

Property-based testing flips the script. Instead of asserting “this specific input gives this specific output,” you assert a property that must hold for *all* inputs, then let the library throw hundreds of randomized cases at it and try to find one that breaks the rule. I'm using `pgregory.net/rapid`.

The generator builds a transaction that is balanced by construction: pick a random number of legs, give each a random amount, then make the last leg the exact negation of everything before it. The sum is zero on purpose. The property: any transaction built this way must pass `Validate`.

```

1 func TestProp_BalancedAlwaysValid(t *testing.T) {
2     rapid.Check(t, func(t *rapid.T) {
3         tx := Transaction{ID: "tx", Postings: genBalancedPostings(t)}
4         if err := tx.Validate(); err != nil {
5             t.Fatalf("balanced transaction rejected: %v", err)
6         }
7     })
8 }

```

Then the mirror image: take a balanced transaction, bump exactly one leg by a non-zero amount, and assert it *always* fails. A balanced ledger you can nudge into imbalance and it still passes would be a disaster.

```

1 func TestProp_PerturbedAlwaysInvalid(t *testing.T) {
2     rapid.Check(t, func(t *rapid.T) {
3         postings := genBalancedPostings(t)
4         idx := rapid.IntRange(0, len(postings)-1).Draw(t, "idx")

```

```
5     delta := rapid.Int64Range(1, 1_000_000).Draw(t, "delta")
6     bumped, _ := NewMoney(postings[idx].Amount.Amount()+delta, "USD")
7     postings[idx].Amount = bumped
8     tx := Transaction{ID: "tx", Postings: postings}
9     if err := tx.Validate(); err == nil {
10         t.Fatal("perturbed transaction passed Validate, expected
                failure")
11     }
12 })
13 }
```

If rapid ever finds a counterexample, it shrinks it down to the smallest input that still breaks and hands it to me. That's the part I love: the test doesn't just say "something's wrong," it says "here is the simplest case that's wrong." For an invariant this important, I want a tireless adversary trying to break it on every run, not just the handful of examples I was clever enough to write down.

Everything passes, the domain package sits at 98% coverage, and the linter (gosec included) is clean.

3.5 What I deliberately left out

No Account balance field. Anywhere. A balance is never stored as mutable state; it's the sum of every posting that ever touched the account, derived when you ask. That's the whole point of double-entry, and storing a balance you also derive is how the two drift apart and lie to you. The Account type this week is identity and classification only: an ID, a name, a type (asset, liability, equity, income, expense), and a currency.

No persistence, no concurrency control, no money math beyond add, subtract, and negate. Those have their own weeks. This week was about getting the shapes right, because every layer above this one (the repository, the service, the API) is going to lean on these types being correct.

3.6 The honest split

Same as last week: I own the decisions, Claude does the mechanical lifting. The choice of int64 minor units over a decimal library, the signed-posting

convention, the idea of making `Validate` the single chokepoint, those are mine and they're written down in ADR-002 so future me can't forget the reasoning.

What Claude did, working from a detailed plan I reviewed first, was the test-driven grind: write the failing test, watch it fail, write the minimal code, watch it pass, commit, repeat. It also caught a real edge case I'd waved off. My `Money.String` formatter negated the amount to print a minus sign, which breaks for the single most-negative `int64` value, whose negation doesn't fit in an `int64`. An absurd amount for a real ledger, but "absurd inputs shouldn't produce nonsense" is exactly the standard this project holds itself to, so we fixed it with an unsigned-magnitude trick and added a test pinning the behavior. Small thing. But small things quietly wrong is the whole genre of bug I'm trying to engineer out of existence.

3.7 Next week

The types are solid and proven. Next week they meet reality: a Postgres schema with append-only postings, a repository layer with `sqlc`, and the first integration test that creates accounts, posts a real transaction, and reads a balance back, all against an actual database spun up in a container.

The repo is open at github.com/sohag-pro/go-ledger. The domain model from this post lives in `internal/domain`, and **ADR-002** explains the money decision in full.

CHAPTER 4

Week 2 Interview Q&A

Interview-style questions for the Week 2 work: the pure internal/domain package (Money, Currency, Account, Posting, Transaction) and ADR-002. Every question is answerable from this repo's code, ADRs, and tests.

4.1 Domain design

4.1.1 1. Walk me through how you modeled a double-entry transaction in Go. What are the core types and how do they relate?

What a strong answer covers:

- Money (signed int64 minor units plus Currency), Account (identity and classification), Posting (one signed Money entry against an account ID), Transaction (an ID plus a slice of Posting).
- A Transaction is two or more Postings; the sign on Posting.Amount carries direction (positive = debit, negative = credit).
- Balance is never a stored field; it is derived by summing postings (see ADR-001).
- The invariant (postings sum to zero) is enforced in one place: Transaction.Validate().

4.1.2 2. Why is Posting.Amount a single signed value instead of an explicit Debit/Credit direction field plus a positive amount?

What a strong answer covers:

- One signed field makes the invariant a plain sum: `Validate` folds the postings with `Money.Add` and checks the result is zero.
- An explicit direction enum would force `Validate` to split sums by side and reconcile them, more code and more room for error.
- The sign convention is documented on the `Posting` type so it is not folklore.
- Tradeoff acknowledged: a direction enum reads more explicitly to an accountant; the signed model was chosen for a simpler, harder-to-break invariant check.

4.1.3 3. Why are `Money`'s fields unexported, and why does construction go through `NewMoney`?

What a strong answer covers:

- Unexported `amount` and `currency` mean a `Money` can only be built through `NewMoney`, which validates the currency shape.
- It keeps `Money` an immutable value type: operations return new values, nothing mutates in place.
- Prevents an invalid `Money` (bad currency) from ever existing in the first place, a “make illegal states unrepresentable” move.
- The accessors `Amount()`, `Currency()`, `IsZero()` give read access without exposing mutation.

4.1.4 4. The zero value of `AccountType` is invalid by design. Explain that choice.

What a strong answer covers:

- The constants start at `iota + 1`, so `Asset` is 1, not 0.
- An uninitialized or zero `AccountType` fails `Validate()` with `ErrInvalidAccountType` instead of silently passing as a real type.
- Defends against the Go footgun where a struct field defaults to a meaningful enum value nobody set on purpose.
- `IsDebitNormal()` and `String()` also treat the zero/undefined value safely (not debit-normal, “unknown”).

4.2 Data integrity and the invariant

4.2.1 5. Where exactly is the balance invariant enforced, and why concentrate it in one place?

What a strong answer covers:

- `Transaction.Validate()` is the single chokepoint: at least two postings, every posting valid, single currency, signed amounts sum to exactly zero.
- One enforcement point means one place to audit, test, and reason about; no scattered checks that can drift.
- It returns the first rule broken via named sentinel errors (`ErrTooFewPostings`, `ErrInvalidPosting`, `ErrCurrencyMismatch`, `ErrOverflow`, `ErrUnbalanced`).
- The domain check is layer one; ADR-001 notes a Postgres CHECK constraint will enforce it again at the database in a later week (defense in depth).

4.2.2 6. Validate never explicitly checks for currency mismatch or overflow, yet it catches both. How?

What a strong answer covers:

- The running sum is built with `Money.Add`, which already returns `ErrCurrencyMismatch` on differing currencies and `ErrOverflow` on int64 overflow.
- `Validate` just propagates the first error from the fold, so the small type does the work and the method stays short.
- This is composition: correctness pushed down into `Money` so every consumer inherits it.
- Demonstrates the value of error-returning arithmetic over panicking or silently wrapping.

4.2.3 7. Why does the ledger derive balances by summing postings instead of storing a balance column you update?

What a strong answer covers:

- A stored mutable balance can drift from the postings that are supposed to explain it; the history would stop being the source of truth.
 - Append-only postings give a built-in audit trail: every balance is reconstructable.
 - A single-sided “add 10 to A” update has no structural guarantee the money came from anywhere; double-entry makes that impossible (ADR-001).
 - Tradeoff: balance reads become aggregations, not single-row lookups; acceptable for v1, and a rebuildable cache can come later if it gets hot.
-

4.3 Money representation (ADR-002)

4.3.1 8. Why int64 minor units for money instead of a float or a decimal library?

What a strong answer covers:

- Floats can’t represent most decimal fractions exactly ($0.1 + 0.2 \neq 0.3$); rounding error accumulates across many postings, unacceptable for money.
- int64 minor units (cents) are exact, allocation-free, and map cleanly onto Postgres BIGINT with no conversion.
- A decimal library (for example shopspring/decimal) is exact too but allocates a big.Int per op and maps to NUMERIC; more machinery than single-currency v1 needs.
- The cost of int64 is overflow risk, handled explicitly in the arithmetic.

4.3.2 9. What is the downside of int64 minor units, and how did you handle it?

What a strong answer covers:

- int64 caps the representable magnitude near $9.2e16$ minor units, far beyond any real balance, but arithmetic can still overflow.
- Add, Sub, Neg are overflow-guarded and return ErrOverflow rather than wrapping or panicking.

- Add detects overflow with sign logic: overflow happened if both operands share a sign but the result's sign flipped.
- Neg special-cases `math.MinInt64`, whose negation does not fit in an `int64`.

4.3.3 10. Your Money carries a Currency even though v1 is single-currency. Why pay that cost now?

What a strong answer covers:

- Cross-currency arithmetic returns `ErrCurrencyMismatch`, so a future multi-currency mistake cannot silently corrupt a sum.
 - It is cheap insurance: the field and the check cost little and make the invariant currency-aware from day one.
 - Multi-currency is explicitly out of scope (ADR-002, build plan), so there is no per-currency scaling logic yet, just the guard.
 - Shows scope discipline: build the cheap guardrail, defer the expensive feature.
-

4.4 Testing

4.4.1 11. You wrote both table-driven tests and property-based tests for the same invariant. Why both, and what does each catch?

What a strong answer covers:

- Table-driven tests pin specific known cases: balanced two-leg and multi-leg, unbalanced, too-few postings, currency mismatch, empty account ID.
- Property tests (pgregory.net/rapid) assert a rule for hundreds of randomized inputs, catching cases the author never imagined.
- Two properties: any balanced transaction passes `Validate`; any one-leg perturbation of a balanced transaction fails.
- `rapid` shrinks a failure to the minimal counterexample, so a regression points straight at the smallest breaking input.

4.4.2 12. Explain the `genBalancedPostings` generator. How do you know it produces genuinely balanced transactions, and what keeps it from triggering false overflow?

What a strong answer covers:

- It draws $n-1$ random legs, tracks the running `int64` sum, then adds a final leg equal to the negation of that sum, so the total is zero by construction.
- A separate property (`TestProp_SumIsZero`) verifies the generator's own output sums to zero.
- Amounts are bounded to roughly $\pm 1e9$ over at most 8 legs, so the running sum stays well within `int64` and the fold never hits a spurious `ErrOverflow`.
- The perturbation delta is always nonzero and within bounds, so a perturbed transaction is genuinely unbalanced, not accidentally rebalanced.

4.4.3 13. The domain package sits around 98% coverage. What is the uncovered slice, and why is that acceptable?

What a strong answer covers:

- The small gap is an error path in `Sub` reached via `Neg` (the `MinInt64` underflow), already exercised indirectly by the overflow test cases.
- Coverage is a signal, not a goal; the invariant itself is covered by both example and property tests.
- Chasing the last percent on an implicitly-tested error branch is low value versus the property coverage already in place.
- Honest about what is and is not exercised rather than gaming the number.

4.5 Trade-offs and defending decisions

4.5.1 14. Defend the double-entry model itself. Why not a simpler balances table with debit/credit helper functions?

What a strong answer covers:

- A mutable balances table has no structural audit trail and no guarantee money came from somewhere; bugs create or destroy money silently (ADR-001 context).
- Double-entry makes an unbalanced transaction rejectable before it persists; the invariant is the schema, not a convention.
- Append-only postings sidestep read-modify-write races on a balance row.
- It matches what accountants, auditors, and payment partners already expect, lowering integration friction.

4.5.2 15. You pulled in `pgregory.net/rapid` in Week 2 even though the plan first names it in Week 9. Justify adding a dependency early.

What a strong answer covers:

- The balance invariant is the single most important property in the system; it deserves adversarial randomized testing from the moment it exists, not seven weeks later.
- `rapid` is a test-only dependency, so it adds zero weight to the production binary.
- Introducing it once and using it consistently beats hand-rolling generators now and swapping later.
- Defensible reversal of a plan detail with a clear reason, which is the point of the question.

4.5.3 16. A `Money.String` edge case came up during review: formatting `math.MinInt64`. What was the bug and how did you fix it without changing the method signature?

What a strong answer covers:

- The original `String` negated the amount to print a minus sign, but negating `math.MinInt64` overflows `int64` and produces nonsense.
- `String` must satisfy `fmt.Stringer`, so it cannot start returning an error; the fix had to stay signature-compatible.
- The fix computes the absolute value as a `uint64` (unsigned negation yields the correct magnitude even for `MinInt64`), with a `//nolint:gosec` and a comment explaining the intentional signed-to-unsigned reinterpretation.
- A test pins the `MinInt64` output so the behavior cannot silently regress; the broader lesson is that absurd inputs still must not produce garbage.

WEEK 3

01 Postgres Schema for a Multi-Tenant Ledger

02 Week 3 Interview Q&A

CHAPTER 5

Postgres Schema for a Multi-Tenant Ledger

Open your banking app and look at the balance. It feels like a saved number, a single field in a row somewhere with your name on it, that goes up when money arrives and down when it leaves. That mental model is wrong, and the gap between that model and how a real ledger works is most of what Week 3 was about.

In a proper ledger your balance is not stored anywhere. It is computed, every time, by adding up every entry that ever touched your account. The history is the truth. The balance is just a question you ask of that history. Last week I built the double-entry domain model in Go types. This week I had to put it on disk, in Postgres, in a way that keeps that property honest. This is post 3 of 12.

5.1 The one decision that shapes the rest

Here is the fork in the road. When someone posts a transaction, you can do one of two things with their balance.

Option A: keep a balance column. Each account row has a balance field. Every posting reads it, adds the amount, and writes it back. Reads are trivial, one column lookup.

Option B: keep only the postings. Postings are append-only rows. The balance is `SUM(amount)` over an account. Nothing is ever updated in place.

Option A is faster to read and it is the one most people reach for first. It is also the exact bug double-entry exists to prevent. The moment you store a balance, you have two copies of the truth: the number in the column, and the sum of the postings. The day those two disagree, and they will, after a crash mid-write, a missed update, a racy retry, you have no way to tell which one is right. Money is now ambiguous, which in a ledger is the worst thing money can be.

So go-ledger uses Option B. Postings are append-only. Balances are derived. There is exactly one source of truth, and it cannot drift from itself.

Picture my dinner debt from post 1, the 600 taka I owed a friend, now sitting in the actual tables:

```

1  postings table (the only truth)
2  +-----+-----+-----+
3  | account  | txn          | amount  |
4  +-----+-----+-----+
5  | sohag    | dinner-debt  | -600    |
6  | friend   | dinner-debt  | +600    |
7  +-----+-----+-----+
8
9  |          |              |          |
10 | SUM(amount) | SUM(amount)
11 | WHERE account=sohag | WHERE account=friend
12 v          v
13 balance: -600      balance: +600
14
15 No balance column anywhere. Ask the question, get the answer.

```

The trade is real and worth saying out loud: reads are now an aggregate, not a lookup. At v1 scale that is a non-issue. If it ever bites, the fix is a cached rollup that is *rebuilt from the postings*, never a mutable primary balance. The history stays sacred.

5.2 Three tables, not five

My build plan listed five tables for this week: accounts, transactions, postings, idempotency_keys, and audit_log. I built three.

The last two belong to Week 6. I do not know their columns yet, because I have not written the code that uses them. Designing a table before its consumer exists just means I get to ALTER it later anyway, so there is no real saving in rushing it. Migrations are cheap and ordered. Each table arrives with the feature that needs it. That is the whole argument, and it is the kind of small discipline that keeps a twelve-week project from quietly turning into a thirty-week one.

So 0001_initial.sql creates accounts, transactions, and postings. That is it.

5.3 Multi-tenant from the first migration

The blog title says multi-tenant, and that is the part most schema posts skip. A real ledger does not hold one person's accounts. It holds many independent tenants' books in the same database, and tenant A must never, under any bug, see tenant B's money.

There is no login or auth in go-ledger yet. So why is there a `tenant_id` on every single table already? Because adding it later is the migration nobody wants to write. Retrofitting tenancy onto a populated ledger means a new column, a backfill across every row, and rebuilt foreign keys, all on live data. Adding the column now, while the tables are empty, costs nothing. The column goes in on day one even though the code that resolves a tenant comes much later.

But a column is just a label. The interesting part is making the database itself refuse to mix tenants. Here is the trick:

```

1  accounts                                transactions
2  +-----+                               +-----+
3  | id      (uuid) |                       | id      (uuid) |
4  | tenant_id (uuid) |                     | tenant_id (uuid) |
5  | ...          |                       | ...          |
6  | UNIQUE (tenant_id,id) |                 | UNIQUE (tenant_id,id) |
7  +-----+                               +-----+
8          ^                               ^
9          | FK (tenant_id, account_id)   | FK (tenant_id,
10         | transaction_id)
11         |                               |
12         +-----+
13         | postings                      |
14         | tenant_id, account_id, transaction_id|
15         +-----+
16
17  A posting must match its account's tenant AND its transaction's tenant.
18  A cross-tenant posting is not "discouraged". It is impossible to insert
19  .

```

Instead of a plain foreign key on `id`, `accounts` and `transactions` carry a `UNIQUE (tenant_id, id)`, and `postings` reference the pair. Now Postgres will reject a posting that points at an account in a different tenant. Tenant isolation stops being a thing every query has to remember and becomes a guarantee the database enforces. The query that forgets its `WHERE tenant_id = ?` is still a bug, but the catastrophic version, a posting that silently links two tenants' books, simply cannot be written.

5.4 The small choices that add up

A few smaller decisions, each with a fork worth a sentence:

UUIDv7 keys, generated in Go. Keys are UUIDs, not auto-incrementing integers, and the app generates them before insert using UUIDv7. Version 7 is time-ordered, so it keeps the nice index locality that makes serial keys fast, while staying globally unique and not coupled to the database. Generating them in Go means the caller knows the ID without a round-trip, and tests are deterministic. The cost is 16 bytes instead of 8. Worth it.

Currency on the transaction, not the posting. The domain already guarantees every posting in a transaction shares one currency. So I store the currency once, on the transaction, and a posting stays a single signed integer amount. Less duplication, and the single-currency rule is visible in the shape of the data.

Account type as text with a CHECK, not a Postgres enum. A column constrained by CHECK (type IN ('asset', 'liability', ...)) reads fine in plain SQL, and adding a type later is an ordinary migration instead of the awkward ALTER TYPE dance an enum forces. Low churn, so the enum's only real benefit does not apply.

No balance CHECK yet. The rule that a transaction's postings must sum to zero lives in the Go domain for now, in `Transaction.Validate()`. The database-level constraint that makes it impossible regardless of caller is deliberately Week 4 work, landing with the concurrency handling where it belongs. I am being honest about that gap rather than pretending the schema is already bulletproof.

5.5 Proving it works against a real Postgres

A schema you have not run is a guess. The Week 3 definition of done was a single integration test that exercises the whole happy path end to end, against a real Postgres, not a mock.

The test uses `testcontainers-go` to spin up an actual `postgres:16-alpine` in a container, runs the migration with `goose`, then does the real thing: create two accounts, post a balanced two-leg transaction, read both balances back, and assert they are +10000 and -10000 and that they net to zero. That last

assertion is the whole point of the project, checked end to end:

```
1 cashBal, _ := repo.Balance(ctx, tenant, cash.ID)      // +10000
2 revBal, _ := repo.Balance(ctx, tenant, revenue.ID)   // -10000
3
4 if cashBal.Amount()+revBal.Amount() != 0 {
5     t.Errorf("ledger does not net to zero")
6 }
```

There is a second test that creates an account under one tenant and tries to read it as another tenant, expecting “not found”. The isolation I designed into the foreign keys is not just a claim in this post, it is a test that fails if I ever break it.

One honest detail: the test skips, rather than fails, when no Docker daemon is reachable, so a teammate without Docker still gets a green `make test`. The risk is that a skip can hide a broken integration path, so CI runs Docker and exercises the real thing for real. On my own laptop that meant getting a container runtime going (I use colima, not Docker Desktop), watching the first run pull the Postgres image, and finally seeing three green `PASS` lines against a live database. That is the moment the schema stopped being a guess.

5.6 The AI companion, week 3

The running subplot of this series is that I build this with Claude Code as a pair, and I stay honest about the split. This week Claude wrote a lot of the mechanical surface: the migration SQL, the sqlc setup, the repository adapter, the integration test. But the decisions in this post, append-only over a balance column, `tenant_id` on day one, composite foreign keys for isolation, three tables not five, are mine, argued out before any code got written and recorded in **ADR-003**. The deal holds: Claude accelerates the typing, I own the why. A ledger I could not explain line by line would defeat the point.

5.7 Where this leaves us

The ledger now has a home on disk that cannot lie to itself. Balances are derived, postings are immutable, tenants are isolated by the database, and there is a test that runs the whole path against real Postgres and proves it.

What is missing is the hard part: what happens when two requests post to the same account at the same time. Right now the balance invariant is enforced in Go, optimistically, with nothing stopping two concurrent writers from racing. Next week is atomicity and concurrency: wrapping posting in a database transaction at `SERIALIZABLE` isolation, adding the database-level `CHECK` that makes an unbalanced transaction impossible, and then pointing 10,000 concurrent posts at the same accounts to see if the invariant holds. That is where ledgers get genuinely interesting.

The repo is open source at github.com/sohag-pro/go-ledger. Follow along or steal the schema.

CHAPTER 6

Week 3 Interview Q&A

Interview-style questions for the Week 3 work: the `0001_initial` migration, the `sqlc` query layer, the `domain.Repository` port and its `internal/postgres` adapter, the `testcontainers` integration test, and ADR-003. Every question is answerable from this repo's code, ADRs, and tests.

6.1 Schema design

6.1.1 1. Walk me through the schema you landed on for Week 3. What are the three tables and how do they relate?

What a strong answer covers:

- accounts (identity, type, currency, tenant), transactions (a single-currency grouping, tenant), postings (signed bigint amount linking a transaction to an account).
- A transaction has two or more postings; balances are not stored, they are `SUM(amount)` over an account (carries ADR-001 into the DB).
- Postings are append-only: no `UPDATE` or `DELETE` query is written against them.
- Every table carries `tenant_id`; the whole ledger is scoped by tenant.

6.1.2 2. You chose append-only postings with derived balances over a mutable balance column. Why?

What a strong answer covers:

- A stored balance is a second copy of the truth that can drift from the postings, and once it drifts there is no way to tell which is right. That is exactly the bug double-entry exists to prevent.

- A mutable balance turns every posting into a read-modify-write that has to be serialized on the account row.
- Derived balance is always reconstructable from history; the posting log is the single source of truth.
- Tradeoff named: reads become an aggregate (SUM) instead of a column lookup. Fine at v1 scale; the escape hatch is a materialized rollup rebuilt from postings, never a mutable primary balance (recorded in ADR-003).

6.1.3 3. The build plan lists five tables but the migration creates three. Defend that.

What a strong answer covers:

- idempotency_keys and audit_log are Week 6 work; their columns are not known until the code that uses them exists.
- Designing a schema before its consumer invites an ALTER later anyway, so there is no real saving from doing it early.
- Migrations are cheap and ordered; adding tables in their own migration keeps each change reviewable and tied to a feature.
- YAGNI applied deliberately, not by accident: the decision is written down in ADR-003.

6.1.4 4. Why does currency live on the transaction and the account, but not on each posting?

What a strong answer covers:

- The domain already guarantees all postings of a transaction share one currency (Transaction.Validate), so storing it per posting would be redundant.
- A posting stays a single signed bigint amount; its Money is reconstructed from the transaction's currency on read (see GetTransaction in repository.go).
- The account also carries its own currency, which is what Balance uses to build the returned Money.
- Keeps the posting row minimal and the single-currency-per-transaction rule structurally visible.

6.2 Multi-tenancy and data integrity

6.2.1 5. There is no auth yet, so why is `tenant_id` on every table from day one?

What a strong answer covers:

- Retrofitting tenancy onto a populated ledger (adding the column, backfilling, rebuilding foreign keys) is a painful, risky migration nobody wants to write later.
- The column and the foreign-key discipline are cheap now and define the scoping seam the service and API will use.
- Repository methods are already tenant-scoped (`tenantID` is the first argument on every method), so the contract is set before any consumer depends on it.
- Auth and tenant resolution land later; the storage shape does not have to wait for them.

6.2.2 6. Explain the composite foreign keys on postings. What do they buy you that a plain FK on `id` would not?

What a strong answer covers:

- accounts and transactions have `UNIQUE (tenant_id, id)`; postings references `(tenant_id, account_id)` and `(tenant_id, transaction_id)`.
- This makes it impossible to insert a posting that points at an account or transaction in a different tenant: the database rejects it, not just application code.
- Tenant isolation becomes a data-integrity guarantee enforced by Postgres, not a convention you hope every query remembers.
- Demonstrated by `TestTenantIsolation`: a second tenant cannot read another tenant's account even with the correct `id`.

6.2.3 7. Account type is stored as text with a `CHECK` constraint instead of a Postgres enum. Why?

What a strong answer covers:

- `CHECK (type IN ('asset','liability','equity','income','expense'))` is readable in plain SQL and queryable without special casts.
- Adding or changing a type is an ordinary migration, versus the awkward `ALTER TYPE ... ADD VALUE` dance (which cannot run in a transaction in older Postgres and cannot remove values).
- The Go side maps the enum with `AccountType.String()` on write and `ParseAccountType` on read, so the text values and the domain enum stay in lockstep.
- Low-churn set, so the enum's main benefit (compact storage) is not worth its rigidity.

6.2.4 8. The schema has no CHECK that postings sum to zero. Isn't that the whole point of the ledger? Where is the invariant enforced?

What a strong answer covers:

- For Week 3 it is enforced in the domain: `CreateTransaction` calls `Transaction.Validate()` before writing anything.
- The database-level CHECK is deliberately Week 4 work, landing with the transaction-posting service and its concurrency handling where it actually belongs.
- Defense in depth is the goal: the domain check catches it early with a typed error; the DB constraint will make it impossible regardless of caller.
- Honest about the gap: until Week 4, a buggy caller that bypassed `CreateTransaction` could write an unbalanced set; that is why the constraint is scheduled, not skipped.

6.3 Repository and adapter design

6.3.1 9. Why is Repository an interface in internal/domain rather than just a struct in the postgres package?

What a strong answer covers:

- Ports and adapters: the domain owns the contract, the infrastructure implements it. Dependencies point inward.
- The domain stays free of pgx and uuid types; the port is a plain Go interface over domain types and context.
- The service layer (Week 4) can be tested against a fake repository without a database.
- A compile-time `var _ domain.Repository = (*Repository)(nil)` keeps the adapter honest about satisfying the port.

6.3.2 10. IDs are uuid in Postgres but string on the domain types. Walk through how that boundary is handled and why UUIDv7 generated in the app.

What a strong answer covers:

- The domain models ids as plain string so it carries no uuid dependency; the adapter parses to `uuid.UUID` at the boundary and stringifies on the way out.
- The app generates UUIDv7 before insert (`uuid.NewV7()`), writing it back to the passed-in pointer when the caller left the id empty.
- v7 is time-ordered, so it keeps b-tree index locality close to a serial key while staying globally unique and decoupled from the DB.
- Generating in Go (not `gen_random_uuid()`) means the caller knows the id without a round-trip and tests are deterministic.

6.3.3 11. How does CreateTransaction guarantee a transaction and its postings are written atomically?

What a strong answer covers:

- It opens a pgx transaction (`pool.Begin()`), runs the transaction insert and every posting insert through `q.WithTx(tx)`, then commits.

- `defer tx.Rollback(ctx)` covers every early-return error path; rollback after a successful commit is a harmless no-op.
- `Transaction.Validate()` runs before any DB work, so an invalid transaction never opens a connection.
- Result: there is no window where a transaction row exists without its postings, or vice versa.

6.3.4 12. Why does Balance read the account first and then sum postings, rather than just running the SUM?

What a strong answer covers:

- The account is the source of the currency, needed to build the returned Money.
 - It distinguishes “account exists but has no postings” (balance zero) from “no such account” (`ErrAccountNotFound`); a bare SUM over zero rows would return 0 and hide a missing account.
 - The query uses `COALESCE(SUM(amount), 0)::bigint` so an account with no postings reads as 0, not NULL.
 - Tradeoff: two round-trips instead of one. Acceptable for correctness; could become a single joined query if it ever mattered.
-

6.4 Testing

6.4.1 13. Describe the Week 3 integration test and how it proves the definition of done.

What a strong answer covers:

- `TestHappyPath` spins a real `postgres:16-alpine` via `testcontainers-go`, runs the goose migration, then creates two accounts, posts a balanced two-leg transaction, and reads both balances back.
- It asserts the specific balances (+10000 and -10000) and that they net to zero, which is the defining double-entry property end to end.

- It also round-trips the transaction through `GetTransaction` and re-runs `Validate()` on the reconstructed value.
- Backed by `TestGetAccountNotFound` and `TestTenantIsolation` for the error and isolation paths.

6.4.2 14. The test skips instead of fails when Docker is absent. Defend that choice and explain the risk.

What a strong answer covers:

- A developer without a running Docker daemon still gets a green `make test`; the suite does not punish them for missing infrastructure.
- The skip is explicit and logged with a clear reason, so it is visible, not silent.
- Risk: a skip can mask a genuinely broken integration path if CI does not run Docker, so CI must run it for real (it does).
- The skip key off the container failing to start, not a `testing.Short()` flag, so the real run needs no special invocation beyond Docker being up.

6.4.3 15. Migrations run through goose in the test but the app talks to Postgres through pgx. Why two database access paths, and how do they coexist?

What a strong answer covers:

- goose is built on `database/sql`, so the test opens a `sql.DB` with the `pgx` `stdlib` driver (`_ "github.com/jackc/pgx/v5/stdlib"`) just to run migrations, then closes it.
 - The repository itself uses a `pgxpool.Pool` (the native `pgx` path) for all real queries.
 - Migrations are embedded with `go:embed` (`postgres.Migrations`) so goose runs them without a path on disk, the same FS the server can use at startup later.
 - Clean separation: schema management and query execution are different concerns and do not have to share a driver.
-

6.5 Trade-offs and stack defense

6.5.1 16. Defend pgx + sqlc over an ORM like GORM. What do you give up?

What a strong answer covers:

- sqlc generates type-safe Go from hand-written SQL, so the queries are explicit, reviewable, and exactly what runs; no hidden N+1 or surprise queries from an ORM's lazy loading.
- pgx gives full access to Postgres features and a fast native protocol path, plus first-class transaction control used directly in `CreateTransaction`.
- For a ledger, predictable SQL and tight control over transactions and isolation matter more than the CRUD convenience an ORM optimizes for.
- Given up: less automatic scaffolding, you write the SQL and the migrations yourself. Accepted as a feature, not a cost, for this domain.

6.5.2 17. Someone argues UUIDv7 primary keys are over-engineering and a bigserial would be simpler and faster. How do you respond?

What a strong answer covers:

- bigserial couples id assignment to the database, forcing a round-trip to learn the id and making cross-tenant uniqueness and external references messier.
- UUIDv7 keeps most of serial's index locality (time-ordered) while being globally unique and client-generatable, which keeps the domain and tests independent of the DB.
- For a multi-tenant system that may later shard or expose ids externally, opaque non-sequential keys avoid leaking volume and ordering and avoid id collisions across tenants.
- Honest cost: 16 bytes versus 8, and slightly larger indexes. Acceptable for the isolation and decoupling gained; documented in ADR-003.

WEEK 4

01 Atomic Transaction Posting in Go

02 Week 4 Interview Q&A

CHAPTER 7

Atomic Transaction Posting in Go

Imagine an account with 100 taka in it, and two withdrawals of 100 taka arriving at the exact same instant. Both read the balance, both see 100, both say “yep, enough money,” and both go through. The account is now at minus 100. Nobody wrote a bug, exactly. Each withdrawal was correct on its own. They were just allowed to happen at the same time, and the system let them both believe they were alone.

That is the whole problem of week 4. Last week I put the ledger on disk and proved a single transaction posts correctly. This week the question is harder: what happens when a hundred of them land on the same accounts at once, and how do I make absolutely sure the books still balance? This is post 4 of 12.

7.1 The thing that must never happen

The ledger’s one rule, from day one, is that every transaction’s postings sum to zero. Money moves, it never appears or vanishes. Under concurrency the failure mode is subtle. It is not that one transaction is wrong. It is that two correct transactions, interleaved, produce a state neither of them intended.

Here is my dinner-debt example from earlier in the series, now with a twist. Say two different transactions both touch my wallet at the same moment: I pay back a friend 600 taka, and at the same time a refund of 200 lands.

1	Transaction A: pay friend	Transaction B: refund arrives
2	sohag:wallet -600	sohag:wallet +200
3	friend:wallet +600	shop:wallet -200
4		
5	Both read sohag:wallet's history at the same instant.	
6	Both append their postings based on what they saw.	
7	If the database lets them interleave wrong...	
8	...the final balance can reflect one, but not the other.	

Both transactions are individually balanced. The danger is in the interleaving. So the real work this week was not writing the posting logic. It was choosing

how the database serializes conflicting transactions, and then proving it holds.

7.2 Two ways to be safe

There are two classic answers, and they are opposites.

The **pessimistic** one: lock the rows. Before touching an account, grab an exclusive lock on it (`SELECT ... FOR UPDATE`), do the work, release. Anyone else who wants that account waits their turn. It works, but correctness now depends on me remembering to lock the right rows, in the right order, on every single code path, forever. The day someone adds a new way to write a posting and forgets the lock, the race is back, silently.

The **optimistic** one: don't lock, just let the database watch. Run every transaction at `SERIALIZABLE` isolation, the strictest level Postgres offers. Postgres tracks the reads and writes of concurrent transactions and, if committing them all would produce a result that no serial (one-at-a-time) ordering could, it aborts one of them with a specific error. Nothing blocks. Conflicts surface as a retry-this error.

I went optimistic. The reason is in the failure mode: with locking, a forgotten lock fails silently and corrupts money. With `SERIALIZABLE`, the database refuses to let an unsafe interleaving commit, no matter what the application code does or forgets. Correctness stops being something I have to remember and becomes something the database guarantees. I wrote this up in **ADR-004**.

The catch is that “abort with a retry-this error” means I have to actually retry.

7.3 Retrying without making it worse

When Postgres aborts a transaction for a serialization conflict, it returns `SQLSTATE 40001`. The fix is to roll back and run the whole thing again, because the second time around the conflicting transaction has usually finished and yours can now commit cleanly.

One subtlety bit me immediately: the conflict often does not show up when you run your statements. It shows up at `COMMIT`. So the retry logic has to watch both the work and the commit:

```
1 attempt:
2 BEGIN SERIALIZABLE
3 insert transaction + postings --> maybe 40001 here
4 COMMIT --> or maybe 40001 here
5 if 40001 anywhere: roll back, wait a bit, try again
```

The “wait a bit” matters more than it looks. If a hundred transactions all conflict and all retry after exactly the same delay, they just collide again, in lockstep, forever. So the backoff is exponential with **full jitter**: each retry waits a random amount up to a growing cap. The randomness is the point. It scatters the retriers so they stop landing on top of each other. Bounded retries, and if they ever run out, the caller gets a typed “conflict, try again” error that the API will map to a 503, not a confusing 500.

7.4 Belt and suspenders: enforce it in the database too

The domain already checks the sum-to-zero rule in Go before anything touches the database. But I wanted the database itself to refuse an unbalanced transaction, so that a bad migration, a direct `psql` session, or some future second service physically cannot write one.

The trouble is that the rule spans many rows. A normal `CHECK` constraint only sees one row at a time, and a single posting of plus 600 is not “wrong,” it is half of a pair. The sum only makes sense once all the postings are in.

The answer is a **deferred constraint trigger**. It is queued up and fires once, at `COMMIT`, after every posting is written, and it checks that each transaction’s postings sum to zero:

```
1 BEGIN
2 insert posting +600 (trigger queued, not checked yet)
3 insert posting -600 (trigger queued, not checked yet)
4 COMMIT
5 --> now the trigger runs: SUM(postings) for this txn = 0 ? yes ->
    commit
6                                     no ->
                                     abort
```

Now the invariant is guaranteed in two independent places: the Go domain catches it early with a clean error, and the database guarantees it absolutely.

There is a test that bypasses all my Go code, inserts a single lopsided posting with raw SQL, and confirms the commit is rejected.

7.5 The afternoon one in six payments failed

Then I wrote the stress test the week was really about: 100 goroutines, 10,000 balanced transactions, all hammering a shared pool of accounts. The bar was zero invariant violations.

The first run was ugly. Roughly **one in six posts failed**, all with the same serialization error, even after retries. Nearly 2,000 transactions gave up. My first instinct was that my retry logic was weak, so I improved the backoff. It got better, not fixed. Still over a hundred failures, and the run took a full minute.

The real cause was somewhere I never would have guessed: a missing index.

My balance trigger reads postings filtered by their transaction id. There was no index on that column, so Postgres did a sequential scan to find them. And here is the part that is easy to miss: under `SERIALIZABLE`, a scan does not just read rows, it takes a broad **predicate lock** over the range it scanned. Two transactions writing completely unrelated postings would both scan, both lock wide swaths of the table, and Postgres would see overlapping read/write footprints and abort one of them as a conflict. The transactions were not actually fighting over the same data. The unindexed read was inventing the conflict.

I added one index on `postings(transaction_id)`. The failures dropped to **zero**, and the run went from about 62 seconds to 5.

1	before: trigger reads postings by transaction_id
2	no index -> sequential scan -> wide predicate lock
3	-> phantom serialization conflicts -> ~1 in 6 posts fail
4	
5	after: same read, now an index lookup -> narrow lock
6	-> real conflicts only -> 10,000 / 10,000 commit, zero failures

That is the lesson I will keep from this week. Under `SERIALIZABLE`, an unindexed read is not merely slow. It actively manufactures conflicts that have nothing to do with your actual data. Indexes are not a performance nicety here, they are load-bearing for correctness-under-load.

One honest footnote on speed: my local numbers were p50 around 35ms, p99 around 68ms, which is above the sub-10ms target I had in mind. The cause is that I run Postgres in a Linux VM on my Mac, and every commit waits for the disk to fsync the write-ahead log through that virtualization layer. I am reporting the real numbers rather than turning off durability to get a prettier graph. For a ledger, the durable write is the entire point.

7.6 What a fresh pair of eyes caught

Before merging, I had my AI pair do a senior-level review of everything through week 4, specifically from a fintech angle. It caught a genuinely good one. Currency lived on accounts and on transactions, but nothing in the database tied them together, so a USD transaction could technically post into a EUR account. Invisible today because everything is USD, but exactly the kind of gap that becomes a real bug the day multi-currency arrives. That became a second small trigger and its own short decision record. The review also pushed the metrics endpoint off the public port (it leaks volumes and latencies) and turned a few raw database errors into typed ones the API can map cleanly.

That is the deal I keep describing in this series: the AI accelerates the work and pressure-tests my thinking, and I own every decision and can explain why each line exists. The currency catch is a good example. It suggested the gap, I decided how to close it and wrote down why.

7.7 Where this leaves us

The ledger now posts transactions atomically, stays correct under heavy concurrency, enforces its core invariant in two independent layers, and has the metrics to watch contention. The stress test runs 10,000 concurrent posts and the books balance to the cent, every time.

What is still missing is a way to actually call any of this over the network. Everything so far is exercised by tests. Next week is the REST API: chi, real endpoints to create accounts and post transactions, proper validation, and RFC 7807 error responses, all sitting on top of the service this week built. That is when the ledger stops being a test suite and starts being something you can curl.

The repo is open source at github.com/sohag-pro/go-ledger. Follow along, or go read your own database's docs on predicate locks before they teach you the hard way.

CHAPTER 8

Week 4 Interview Q&A

Interview-style questions for the Week 4 work: the `TransactionService`, the `RunInTx SERIALIZABLE` retry loop, the deferred balance trigger and the per-account currency trigger, the Prometheus metrics, the 10,000-post stress test, and ADR-004 and ADR-005. Every question is answerable from this repo's code, ADRs, and tests.

8.1 Concurrency model

8.1.1 1. Walk me through what happens when two requests post transactions that touch the same account at the same time.

What a strong answer covers:

- Each `Post` runs in its own `SERIALIZABLE` transaction via `RunInTx`; Postgres's SSI tracks read/write dependencies between them.
- If committing both would violate serial-equivalent ordering, Postgres aborts one with `SQLSTATE 40001`, often only at `COMMIT` time.
- The adapter catches that, rolls back, waits a jittered backoff, and replays the whole unit of work; the loser eventually commits once the winner is done.
- The ledger is never left unbalanced: a conflicting transaction either commits in full or not at all.

8.1.2 2. Why `SERIALIZABLE` plus retry instead of `READ COMMITTED` with `SELECT FOR UPDATE`?

What a strong answer covers:

- `SERIALIZABLE` makes correctness the database's job; the application cannot forget to lock the right rows in the right order, which is the failure mode of pessimistic locking (ADR-004).
- Pessimistic locking serializes access to hot accounts by blocking; optimistic lets non-conflicting work proceed and pays only for actual conflicts.
- The trade is that conflicts cost retries rather than blocked connections, and `fn` must be safe to run more than once.
- Defends against the “what if someone adds a new write path” risk: a future code path can't silently skip a lock it never knew about.

8.1.3 3. Serialization conflicts often surface at `COMMIT`, not mid-transaction. How does `RunInTx` account for that?

What a strong answer covers:

- The retry loop watches both the result of `fn` and the result of `tx.Commit`; either returning `40001/40P01` triggers a replay.
- A naive loop that only checks `fn`'s error would miss the common case and leak the failure to the caller.
- `isSerializationFailure` classifies via `pgconn.PgError` codes (`40001` serialization failure, `40P01` deadlock), the two worth retrying.
- On a non-retryable error it returns immediately; there is a test (`TestRunInTxNonRetryablePropagates`) that pins this.

8.1.4 4. Your retry backoff is exponential with full jitter. Why the jitter specifically?

What a strong answer covers:

- Without jitter, a crowd of transactions that conflicted will all back off the same amount and retry in lockstep, colliding again, a thundering herd.
- Full jitter (`retryBackoff` returns a random duration in `[0, exp]`, capped) de-synchronizes retriers so they spread out and clear.
- Backoff is bounded (25 attempts, cap 100ms) and respects context cancellation while waiting.
- Exhaustion returns a typed `domain.ErrConflict`, not a raw pg error, so transport can map it to 503.

8.2 Enforcing the invariant in the database

8.2.1 5. The balance invariant is already in the Go domain.

Why also enforce it in Postgres, and how?

What a strong answer covers:

- Defense in depth: the domain gives a fast, clean error, but the database guarantees the invariant holds against a bad migration, a direct psql write, or a second service (ADR-004).
- A row-level CHECK can't express it because the sum spans many posting rows.
- Migration 0002 uses a DEFERRABLE INITIALLY DEFERRED constraint trigger that fires at COMMIT, after all postings are in, and rejects any transaction whose postings don't sum to zero.
- Proven by TestUnbalancedRejectedByTrigger, which inserts raw unbalanced rows and asserts COMMIT fails.

8.2.2 6. Why DEFERRABLE INITIALLY DEFERRED rather than a trigger that fires on each insert?

What a strong answer covers:

- A transaction is built from multiple posting inserts; after the first insert the set is not yet balanced, so an immediate per-row check would reject every valid transaction.
- Deferring to COMMIT means the check sees the complete set, regardless of how many statements inserted the rows.
- It is independent of insert order and of which client wrote the rows.
- The trade: it is FOR EACH ROW (constraint triggers must be), so it runs N times for an N-leg transaction; N is tiny, so the repeated SUM is negligible, and the migration comments say so to stop a wrong "optimization."

8.2.3 7. (Hard) The stress test went from roughly 1 in 6 posts failing to zero failures after one change. What was it, and why did it matter so much?

What a strong answer covers:

- The balance trigger reads postings by `transaction_id`, and there was no index on that column, so the read was a scan.
- Under SSI, a scan takes broad predicate locks, which manufacture false-positive serialization conflicts between transactions that don't actually conflict.
- Adding `postings_transaction_idx` (migration 0002) made the trigger's read a narrow index lookup, collapsing the conflict rate to zero and cutting runtime from about 62s to 5s.
- The lesson: under `SERIALIZABLE`, an unindexed read is not just slow, it actively creates conflicts. Indexes are load-bearing for concurrency, not just performance.

8.2.4 8. You added a second trigger for currency. What gap did it close?

What a strong answer covers:

- Currency lives on accounts and on transactions, but postings carry only an amount, so nothing tied a transaction's currency to each account's currency (ADR-005).
- Without it, a USD transaction could post into a EUR account; invisible at single-currency v1, but a latent integrity hole.
- Migration 0003 adds an immediate `AFTER INSERT` trigger that rejects a posting whose account currency differs from its transaction currency.
- It is immediate (not deferred) because the account and transaction already exist at insert time, and its reads are primary-key equalities, so the SSI footprint is negligible. Covered by `TestCurrencyMismatchRejectedByTrigger`.

8.3 Architecture and error handling

8.3.1 9. Where does the transaction boundary live, and why is the service thin?

What a strong answer covers:

- The domain owns a Tx unit-of-work interface and `Repository.RunInTx`; the adapter owns isolation and retry; the service composes the work inside `RunInTx`.
- `TransactionService.Post` validates up front, times the operation, records metrics, logs, and maps errors; it holds no SQL and no isolation knowledge.
- This keeps the `SERIALIZABLE`/retry detail in one place (the adapter, which knows pg error codes) and makes the service the single entry point REST and gRPC will both call.
- `RunInTx` is a general seam, so Week 6 can write the audit-log row inside the same transaction as the posting.

8.3.2 10. How do callers tell a transient conflict apart from a real failure, and why does that distinction matter for an API?

What a strong answer covers:

- Retry exhaustion returns `domain.ErrConflict` (transient), wrapping the `SQL-STATE` for logs; a duplicate id returns `domain.ErrDuplicateTransaction`.
- A transport layer maps `ErrConflict` to 503 (retry me) and a validation failure to 4xx, not a blanket 500.
- Returning a typed error instead of a raw postgres: ... string means callers match on errors.Is, not on text.
- An unbalanced transaction returns the domain error unchanged from `Post`, so it becomes a 4xx, not a server error.

8.3.3 11. You validate the transaction in the service, but not inside the retried unit of work. Walk through that decision.

What a strong answer covers:

- Validation happens once at the public entry points (`TransactionService.Post`

and the convenience `Repository.CreateTransaction()`, before the transaction starts.

- `txRepo.CreateTransaction` trusts that `t` is already valid, so validation does not re-run on every retry.
- Fast-fail: an unbalanced transaction never opens a database connection.
- The cost is that `txRepo` is an internal type with a documented precondition rather than a defensively-validating public one.

8.4 Observability and operations

8.4.1 12. What did you instrument, and where is the metrics endpoint exposed?

What a strong answer covers:

- A histogram `transaction_post_duration_seconds` labeled by outcome, and a counter `transaction_post_serialization_retries_total` so contention is visible, plus standard `go_/_process_` collectors.
- The scrape endpoint runs on a separate server bound to loopback (`127.0.0.1:9090`), not the public API port, because metrics leak volumes, latencies, and retry rates (ADR-004).
- The public liveness check stays `GET /healthz`; in production `/metrics` returns 404 on the public interface.
- A scraper reaches metrics over the private network or an SSH tunnel.

8.4.2 13. (Hard) Your stress test runs 100 goroutines but you say the real concurrency is lower. Explain.

What a strong answer covers:

- The true ceiling on simultaneous transactions is the connection pool's `MaxConns`, not the goroutine count; extra goroutines block on pool acquisition.
- The test sets `MaxConns` to 25 explicitly and logs it, so “100 goroutines” means up to 25 transactions truly concurrent at Postgres (ADR-004).
- Quoting the goroutine count would overstate the contention; the honest number is the pool cap.

- `NewPool` also sets statement, lock, and idle-in-transaction timeouts so a stuck operation can't hold a connection indefinitely.

8.5 Testing

8.5.1 14. How do you test the retry logic deterministically, given real serialization conflicts are racy?

What a strong answer covers:

- `RunInTx` takes a function, so a test can pass an `fn` that returns a synthetic `pgconn.PgError{Code: "40001"}` on the first call and `nil` after, exercising `retry-then-commit` without a real conflict.
- `TestRunInTxRetriesThenCommits` asserts exactly two attempts and that the retries counter moved by one; `TestRunInTxExhaustionReturnsConflict` asserts a persistent conflict ends as `ErrConflict`.
- The full stress test still exercises real contention end to end; the unit tests pin the edges the stress test hits only incidentally.
- These lifted the adapter's coverage without depending on flaky timing.

8.5.2 15. (Hard) The integration tests passed locally but failed in CI with “connection reset by peer.” What happened and how did you fix it?

What a strong answer covers:

- The container helper waited only for port 5432 to be listening, but Postgres opens that port during `initdb` and then restarts it, so the port-open signal precedes real readiness.
- Locally on a slow, serial container runtime it never raced; in CI, parallel container startup hit the gap and `goose` got connection resets.
- Fix: wait on the log line “database system is ready to accept connections” with `WithOccurrence(2)` for the `initdb`-then-server double start.
- Broader point: container readiness is about the service accepting queries, not a TCP port being open; the test's correctness depended on the wait strategy.

8.6 Defending the stack

8.6.1 16. Someone argues **SERIALIZABLE** is too slow and you should drop to **READ COMMITTED** for throughput. How do you respond?

What a strong answer covers:

- For a ledger, correctness under concurrency is the product; **READ COMMITTED** would let anomalies (lost updates, write skew) through that double-entry exists to prevent.
- The measured cost is retries under contention, and with the right index plus jittered backoff the test commits all 10,000 posts with zero failures.
- If a specific hot path ever needs it, the optimization is a narrower one (better indexing, batching, or targeted **FOR UPDATE**), not weakening isolation globally.
- The latency baselines (p50 about 35ms, p99 about 68ms on a VM) are dominated by WAL fsync, not isolation, and durability is non-negotiable for money.

8.6.2 17. Defend **pgx** over **database/sql** here, given Week 4 leans on transaction control.

What a strong answer covers:

- `RunInTx` needs explicit `BeginTx` with `pgx.Serializable`, typed error codes via `pgconn.PgError`, and a pool it can tune (`NewPool` sets `MaxConns` and `timeouts`); `pgx` exposes all of this cleanly.
- The `sqlc`-generated queries bind to the `pgx` transaction via `WithTx`, so the same typed queries run inside or outside a transaction.
- `database/sql` would abstract away the Postgres-specific error codes and some pool controls the retry logic relies on.
- `goose` still uses `database/sql` for migrations via the `pgx` `stdlib` driver, so each tool is used where it fits.

WEEK 5

01 Designing a REST API for a Payment Ledger

02 Week 5 Interview Q&A

CHAPTER 9

Designing a REST API for a Payment Ledger

Open your banking app and scroll the statement. Every row has a little running balance on the right: after this coffee you had 4,210, after that salary you had 54,210, and so on down the list. It feels like each of those numbers was saved next to the transaction when it happened. It was not. That number does not exist anywhere until you ask for it, and producing it correctly, in pages, fast, turned out to be the most interesting thing I built this week.

For four weeks go-ledger has been a thing that only its own tests could talk to. This week I gave it a REST API and put it online. As of now you can actually curl it at `go.sohag.pro`. This is post 5 of 12.

9.1 Resources and verbs, not remote procedures

A ledger has a small, sharp set of nouns: accounts and transactions. So the API is boring in the good way, just those resources and the standard verbs:

1	POST /v1/accounts	create an account
2	GET /v1/accounts/{id}	read one
3	GET /v1/accounts/{id}/balance	its current balance
4	GET /v1/accounts/{id}/statement	its postings, with running balance
5	POST /v1/transactions	post a balanced transaction
6	GET /v1/transactions/{id}	read one

People sometimes ask why not GraphQL, since it is the fashionable default for a new API. GraphQL earns its keep when clients need to shape wildly different views over a big, interconnected graph, and when over-fetching is a real problem. A ledger is the opposite of that. The resource set is tiny and well defined, the relationships are obvious, and the queries that matter (“what is this account’s balance”, “show me this account’s statement”) are known up front. GraphQL would buy me a flexible query language I do not need and hand me $N+1$ queries and caching headaches I would rather not have. REST over JSON maps cleanly onto these resources, it is curlable and cacheable, and everyone already understands it. gRPC is still coming later for internal

service-to-service calls, sharing the same core, but the public front door is REST.

9.2 Money on the wire

Here is the first decision that matters for a payment API: how does an amount look in JSON? go-ledger sends money as a signed integer count of the smallest unit, plus a currency code:

```
1 { "amount": 10000, "currency": "USD" }
```

That is 100.00 dollars, as 10,000 cents. No decimal point, no floating point, no string parsing at the edge. It maps one to one onto the `int64` the ledger uses internally, so the API is exactly as precise as the ledger is. This is the same choice Stripe and most payment APIs make, and for the same reason: the moment you let `100.00` become a float somewhere, you have invited rounding error into a place where money lives. A debit posting is a positive amount, a credit is negative, and a transaction's postings still have to sum to zero. Posting that coffee sale looks like this:

```
1 POST /v1/transactions
2 {
3   "currency": "USD",
4   "postings": [
5     { "account_id": "...cash...", "amount": 450, "description": "
      latte" },
6     { "account_id": "...revenue...", "amount": -450, "description": "
      latte" }
7   ]
8 }
```

Two legs, plus 450 and minus 450, summing to zero. The API hands the request to the same domain code and the same database triggers from the earlier weeks, so all the invariants still hold. The HTTP layer is thin on purpose: it translates JSON to domain types, calls a service, and maps any error to a clean status code. An unbalanced transaction comes back as a 422, a missing account as a 404, a write conflict as a 503, all as RFC 7807 `problem+json`.

9.3 The statement, and the number that is not stored

Back to that running balance. A statement is a list of the postings that touched one account, newest first, and next to each one, the balance of the account as of that posting. Remember the rule from week 3: balances are never stored, they are summed from the postings. So the running balance on row N is the sum of every posting up to and including row N.

1	postings for "Cash", oldest to newest	running balance
2	+-----+	(cumulative sum)
3	+5000 opening float ----->	5000
4	-450 latte ----->	4550
5	+1200 pastry sale ----->	5750
6	-450 latte ----->	5300
7	+-----+	
8	the statement shows these newest first,	
9	each carrying its own running total	

In SQL, that running total is a window function: `SUM(amount)OVER (ORDER BY created_at, id)`. It sums the postings in order, carrying the total forward row by row. Easy enough, until you remember the statement is paginated. A page shows maybe 50 rows out of an account's thousands. If I compute the window over just the 50 rows on the page, the running balances are wrong, they restart from that page instead of from the beginning of the account's history. The window has to see the whole history even when I only return a slice of it. So the query computes the running balance over all of the account's postings first, then slices out one page.

That is honestly expensive: each page scans the account's full history to rebuild the running totals, because I refuse to store a balance anywhere (storing one is exactly the drift bug double-entry exists to prevent). At this project's scale it is a non-issue, and the index from week 4 makes the scan a tight range. If it ever becomes a problem, the fix is a cached rollup rebuilt from the postings, never a mutable stored balance. The history stays the only truth.

9.4 Paging without lying to the user

The other half of statements is how you ask for the next page. The obvious way is "limit 50, offset 50, offset 100", and it is also subtly broken. If someone

posts a new transaction while you are paging, every offset shifts by one, and you either see a row twice or miss one entirely. For a bank statement, silently skipping a line is not a great look.

So go-ledger uses keyset pagination. Instead of “skip 50 rows”, each page hands back an opaque cursor that points at the exact last row it showed, and the next page asks for “everything older than this row”:

1	page 1: newest ... down to row X	-> cursor = position of row X
2	page 2: everything strictly older than row X	-> cursor = row Y
3	page 3: ...older than row Y...	-> no more rows, cursor = null

The cursor is the row’s (created_at, id) pair, base64-encoded so the client treats it as opaque. Because it anchors on a real row rather than a count, new inserts at the top do not shift anything underneath you. Pages stay stable. When a page comes back shorter than the limit, that is the end, and the response says next_cursor: null.

9.5 When the query generator pushed back

A small war story. I write SQL by hand and use sqlc to generate type-safe Go from it. My first version of the statement query put the window function in a subquery and used a tuple comparison, (created_at, id) < (cursor), for the keyset. Postgres is perfectly happy with that. sqlc was not: its analyzer could not resolve the subquery alias and refused to generate. I tried expanding the comparison out longhand. Still no. What finally worked was moving the window into a CTE (a WITH block) with plain unqualified columns.

The lesson stuck with me: sqlc has its own SQL analyzer, separate from Postgres, and a query the database accepts can still need restructuring for the generator to understand it. That is the tax for type-safe queries with no ORM in the middle, and I will pay it, but it is worth knowing the tax exists. I cover the actual behavior with an integration test against a real Postgres, because sqlc only proves it can parse and type the query, not that it returns the right rows.

9.6 The docs cannot drift

One thing I am quietly proud of: I never write the OpenAPI spec by hand. The API is built with huma, where each endpoint is a typed Go function, and the spec is generated from those same functions. There is a test that fails the build if the committed spec does not match what the handlers produce. So the interactive docs at /playground and the spec at /openapi.json are always exactly the running API, not a hopeful description of it that rots over time. Adding an endpoint and forgetting to document it is not possible here, the test catches it.

9.7 It is real now

The quiet milestone this week is that go-ledger stopped being a test suite. For four weeks the server ran with no database at all, deliberately, because nothing it served needed one. This week the endpoints need real persistence, so the server now connects to Postgres, runs its own migrations on startup, and refuses to boot without a database, because a ledger API with no ledger behind it should not pretend to work.

That also meant standing up Postgres on the actual server for the first time: a native install, a dedicated database and user listening only on localhost, tuned down to fit a 1 GB box, with a nightly backup. Then I pushed, the pipeline built and deployed, and a few seconds later I created an account and posted a transaction against the live API and watched the balance come back correct. The thing works in the world now, not just on my laptop.

9.8 The AI companion, week 5

Same arrangement as every week: I build this with Claude as a pair and stay honest about the split. This week it wrote a lot of the mechanical surface, the handler structs, the error mapping, the fake repository for the HTTP tests, and it was genuinely useful when sqlc started rejecting my query, working through the CTE rewrite with me. The shape of the API, the choice to avoid GraphQL, money as integer minor units, keyset over offset, the decision to keep balances derived even when it makes statements more expensive, those were mine,

argued out and written down in **ADR-006**. The deal holds: it accelerates, I decide and understand.

9.9 Where this leaves us

The ledger has a front door now. You can create accounts, post transactions, read balances, and pull a paginated statement with a running balance, all over a clean REST API with generated docs, live on the internet.

What it does not have yet is safety against the thing that haunts every payment API: the retry. A client posts a payment, the network drops the response, the client retries, and now you have charged someone twice. Next week is idempotency keys and an immutable audit log, the two patterns that make a payments API boringly safe to call. That is where this gets properly fintech.

The repo is open source at **github.com/sohag-pro/go-ledger**. Go curl it, or go read why your favorite API uses keyset pagination and never told you.

CHAPTER 10

Week 5 Interview Q&A

Interview-style questions for the Week 5 work: the v1 REST API built with huma on a chi router, the account statement endpoint (running balance + keyset paging), per-posting descriptions, money on the wire, the default-tenant seam, error mapping, the server's database wiring with startup migrations, and ADR-006. Every question is answerable from this repo's code, ADRs, and tests.

10.1 API design

10.1.1 1. Walk me through the v1 API surface and how a request flows from chi to the database.

What a strong answer covers:

- chi is the router with a middleware chain (request id, recovery, slog request logging); huma rides on it via the humachi adapter and owns operations, validation, and error formatting.
- A request hits a typed huma operation (input struct validated from tags), the handler calls an application service (AccountService/TransactionService), which calls the repository port, which runs sqlc queries on pgx.
- Six endpoints: create/get account, balance, statement, create/get transaction.
- The transport is thin: handlers translate between JSON and domain types and map errors; no SQL or business rules live there.

10.1.2 2. huma generates the OpenAPI spec from the handlers. Why did you choose that over hand-writing the spec or using go-playground/validator plus a separate generator?

What a strong answer covers:

- One source of truth: validation, problem+json errors, and the spec all come from the same typed handler, so the published docs cannot drift from the running API (there is a drift test that fails CI if `api/openapi.yaml` is stale).
- The build plan's literal wording (go-playground/validator, separate generator) predates the huma decision; huma's tag validation supersedes it and removes three moving parts.
- chi stays the router, so the locked decision holds; huma is a layer on top, not a replacement.
- Cost named: huma owns request decode/encode and uses its own validation tag dialect, so very custom responses mean working within its model.

10.1.3 3. How is money represented on the wire, and why?

What a strong answer covers:

- A signed integer count of minor units plus a currency code (`{"amount": 10000, "currency": "USD"}`), mapping 1:1 to the internal `int64`.
- No floating point and no decimal string parsing at the boundary, so the API is as exact as the ledger.
- This is the common payment-API convention (Stripe and others).
- The alternative (decimal strings) was rejected: it adds parsing and per-currency scale validation for no gain (ADR-006).

10.1.4 4. There is no auth yet, but the schema is multi-tenant. How does a request get a tenant?

What a strong answer covers:

- A single default tenant (`DEFAULT_TENANT_ID`, with a fixed fallback) is injected by the API layer into every service call; tenancy is not in URLs or a client header.

- When auth lands, it populates the same tenant value from a token instead, with no change to any resource URL.
- Rejected alternatives: a trusted X-Tenant-ID header (unauthenticated, spoofable) or tenant-in-path (bakes tenancy into the resource hierarchy).
- The repository methods are already tenant-scoped, so the seam is in place.

10.2 Account statement

10.2.1 5. Explain the statement endpoint: what it returns and how it paginates.

What a strong answer covers:

- GET /v1/accounts/{id}/statement returns the account's postings, newest first, each with the running balance as of that posting, plus an optional next_cursor.
- Paging is keyset (cursor) on (created_at, id): the cursor is an opaque base64 token of the last entry's position, and the next page returns entries strictly older than it.
- A full page hands back a cursor; a short page returns next_cursor: null, signaling the end.
- Keyset paging is stable under concurrent inserts in a way offset paging is not.

10.2.2 6. Why keyset pagination instead of limit/offset?

What a strong answer covers:

- Offset can skip or duplicate rows when new postings are inserted mid-paging; keyset anchors on a real row position, so pages stay stable.
- Keyset is also cheaper at depth: no scanning past N rows to reach the offset.
- The cursor encodes (created_at, id); id (the posting id) is the unique tiebreaker, so two postings sharing a timestamp still page deterministically.
- The trade is a slightly more complex cursor and that you page forward in one direction.

10.2.3 7. (Hard) The running balance needs every prior posting, but a page returns only N rows. How do you compute it correctly, and what does that cost?

What a strong answer covers:

- The query is a CTE that computes `SUM(amount)OVER (ORDER BY created_at, id)` over the account's full posting history, then a keyset filter and limit select one page from it.
- A window function applied only to the page's rows would compute a per-page sum, not the absolute running balance; the window must see the whole history.
- Because balances are derived and never stored (ADR-003), this scans the account's history per page, served by the `(tenant_id, account_id)` index.
- It is $O(\text{history})$ per page, fine at v1 scale; the future optimization if it ever bites is a materialized running balance rebuilt from postings, never a mutable primary balance.

10.2.4 8. (Hard) sqlc rejected your first statement query. What was the issue and how did you resolve it?

What a strong answer covers:

- The first version used a windowed derived subquery aliased `s` and a row-value tuple comparison `(s.created_at, s.id)< (...)` in the WHERE; sqlc's analyzer could not resolve the alias there.
- Rewriting the keyset as an expanded comparison did not help; moving the window into a CTE with unqualified columns did.
- The lesson: sqlc uses its own analyzer, so some valid Postgres (windowed derived tables, row-value comparisons) needs restructuring to generate.
- The behavior is verified by an integration test against real Postgres, since sqlc only checks that it can parse and type the query.

10.3 Data and error handling

10.3.1 9. You added a per-posting description. Where is it validated and bounded?

What a strong answer covers:

- `domain.Posting` carries an optional `Description`, bounded at `MaxPostingDescriptionLen` (256) and checked in `Posting.Validate`.
- Migration 0004 adds the column with a matching `CHECK (char_length(description) <= 256)`, so the database enforces the same bound.
- It appears on every posting in requests and responses, including statement entries.
- Defense in depth, consistent with how the other invariants are enforced in both the domain and the database.

10.3.2 10. How do domain errors become HTTP responses?

What a strong answer covers:

- A single `toHumaErr` mapper turns domain errors into huma status errors, which render as RFC 7807 `application/problem+json`.
- Not-found to 404, duplicate id to 409, an exhausted serialization conflict (`ErrConflict`) to 503, validation and invariant failures (unbalanced, currency mismatch, too few postings, bad description) to 422, and anything unrecognized to a 500 that does not leak internals.
- Returning the domain error unchanged from the service lets the handler map it; an unbalanced transaction becomes a 422, not a server error.
- This keeps error semantics in one place rather than scattered across handlers.

10.3.3 11. The plan listed five endpoints; you shipped six. Why, and is that scope creep?

What a strong answer covers:

- The sixth is the account statement, a core ledger feature (listing a single account's entries with a running balance); a ledger without one is hard to defend.
- It reuses the existing schema and the derived-balance model, so it is a natural extension, not a new subsystem.

- It is bounded: keyset paging, a max page size, and the same tenant and currency handling as the rest.
- Knowing when an addition is in the spirit of the system versus genuine creep is the judgment being shown.

10.4 Server and operations

10.4.1 12. The server now depends on Postgres. How are migrations applied, and why that way?

What a strong answer covers:

- `cmd/server` runs the embedded goose migrations on startup, before serving, using a short-lived database/sql handle over the `pgx` `stdlib` driver.
- On a single instance this is the simplest correct option: the binary that needs a column also creates it, atomically with the deploy.
- It does not scale to multiple instances (they would race); the upgrade path is moving the same `goose.Up` into a CI step.
- `DATABASE_URL` is required and the server fails fast without it: a ledger API with no database must not serve.

10.4.2 13. Walk me through how the server is wired now: router, services, and the two HTTP servers.

What a strong answer covers:

- `run` loads config, applies migrations, opens the tuned pool (`NewPool`, bounded conns + timeouts), builds the account and transaction services, and constructs the chi router.
- The router mounts the landing page, the Scalar playground, and the huma API (`/v1/*`, `/healthz`, `/openapi.*`); metrics stay on a separate loopback server (ADR-004).
- Middleware: request id, recovery, and a structured slog logger that records method, path, status, size, duration, and request id.
- `RealIP` is intentionally omitted because it trusts spoofable forwarding headers without a trusted-proxy allowlist.

10.4.3 14. (Hard) How did you test the API without standing up a database for every test?

What a strong answer covers:

- Handler tests run against an in-memory `fakeRepo` that implements the `domain.Repository` port, so the full HTTP path (routing, validation, error mapping, serialization) is exercised with no database.
- The fake covers the happy path and every 4xx: bad enum/pattern to 422, missing resource to 404, unbalanced to 422, plus statement pagination with running balances.
- The real database paths (the statement window query, the balance trigger, concurrency) are covered separately by testcontainers integration tests.
- This keeps the handler tests fast and deterministic while still proving the SQL against real Postgres elsewhere.

10.5 Defending the stack

10.5.1 15. Why a REST API at all, and not gRPC or GraphQL for the public surface?

What a strong answer covers:

- REST over JSON is the lowest-friction public surface: curlable, cacheable, universally understood, and it maps cleanly to the ledger's resources (accounts, transactions) and verbs.
- gRPC is still coming for internal/service-to-service use (a later week), sharing the same domain core; the two are not exclusive.
- GraphQLs flexible query graph buys little for a small, well-defined resource set and adds $N+1$ and caching complexity; the API avoided it deliberately (the blog title says as much).
- The OpenAPI spec and playground give REST much of the discoverability people reach to GraphQL for.

10.5.2 16. Defend pgx + sqlc again now that there is a window function and keyset pagination in play.

What a strong answer covers:

- sqlc generates type-safe Go from hand-written SQL, so the window/CTE statement query is explicit and reviewable, and its parameters and row type are checked at generate time.
- When sqlc could not analyze the first query shape, the SQL was restructured rather than hidden behind an ORM that might have generated something slower or surprising.
- pgx gives the typed error codes and pool tuning the rest of the system relies on (serialization retries, timeouts).
- An ORM would abstract the exact SQL away, which is the opposite of what a ledger wants for its read and write paths.

WEEK 6

- 01 Idempotency Keys and the Immutable Audit Log
- 02 Week 6 Interview Q&A

CHAPTER 11

Idempotency Keys and the Immutable Audit Log

You are paying for something on your phone, you tap Pay, and the spinner just sits there. No confirmation, no error, nothing. Did it go through? Your thumb hovers over the button. Tap it again and maybe you pay twice. Do nothing and maybe you did not pay at all. We have all been in that half second of doubt, and the reason most of us tap again without really panicking is that somewhere a backend engineer already solved this for us. The second tap does not charge you twice. That property has a name, idempotency, and this week I built it into go-ledger, along with its close cousin: an audit log that can record what happened and never let anyone quietly rewrite it.

This is post 6 of 12. Last week the ledger got a REST API and went live at go.sohag.pro. This week is about making that API safe to retry and easy to trust.

11.1 The retry problem is not rare, it is the default

Here is the uncomfortable truth about networks: a request that times out did not necessarily fail. Your `POST /v1/transactions` left the phone, reached the server, posted the transaction, and then the response got lost on the way back. From the client's point of view that looks identical to a request that never arrived. So the client retries, because retrying is the correct thing to do when you are not sure. And now the server sees the same payment twice.

Without protection, that is two transactions, two movements of money, one angry customer. The fix is an idempotency key: the client generates a unique string once per intended action and sends it on every retry of that action.

```
1 POST /v1/transactions
2 Idempotency-Key: 7f3a9c2e-pay-dinner-share
3 { "currency": "BDT", "postings": [ ... -600 ... +600 ... ] }
```

The contract is simple to state and surprisingly interesting to build:

- First time the server sees this key: do the work, remember the result.
- Same key again, same request: do not do the work again, return the original result.
- Same key, but a different request body: refuse, because the client is confused and reusing a key for two different things is a bug worth shouting about (HTTP 409).

11.2 The naive version has a hole in it

The obvious implementation is: look up the key in a table, if it is there return the saved response, if not then post the transaction and save the response. Read, then decide, then write.

That works fine until two copies of the same request arrive at nearly the same instant, which is exactly what a double tap or an aggressive retry produces. Both requests look up the key. Neither finds it, because neither has finished yet. Both decide “new key, go ahead.” Both post the transaction. You are back to charging twice, except now it only happens under load, which is the worst kind of bug because your tests never see it.

The check and the write have to be one atomic step, and the database is the only thing in this picture that can make that guarantee. So instead of checking first, I let the database’s uniqueness constraint be the referee. The idempotency key row goes into a table whose primary key is the tenant plus the key. Inserting it inside the same database transaction that posts the ledger transaction means the whole thing commits together or not at all:

```

1      100 identical requests, same Idempotency-Key
2      |
3      +-----+-----+-----+ ... +-----+-----+
4      |         |         |         |         |
5      v         v         v         v         v
6      [ each opens a SERIALIZABLE transaction and tries: ]
7      insert transaction + postings
8      insert idempotency_key (tenant, key)  <-- unique, this is the
          referee
9      insert audit_log row
10     commit
11     |
12     the FIRST to commit the key wins ----> commits everything
13     |
14     the other 99 hit a duplicate key ----> whole transaction
15                                     rolls back,

```

```
16                                nothing  
16                                they wrote  
16                                survives  
17                                |  
18                                v  
19    losers look up the winner's row and REPLAY its response  
20                                |  
21                                v  
22    result: 1 transaction, 1 audit row, 99 replays
```

The key insight is that the losing requests do not just get rejected. When a request loses the race, its entire transaction rolls back, so the transaction and postings it optimistically wrote vanish too. Then it reads back the row the winner committed, loads that original transaction, and returns it as if it had done the work. From the client's side, all 100 retries got a clean, identical answer. Under the hood, money moved exactly once.

To make the “same key, different body” case work, the stored row also holds a fingerprint of the original request: a hash of the currency and every posting. On a repeat, if the incoming request hashes to the same fingerprint it is a genuine retry and gets the replay. If it hashes differently, someone reused a key for a different payment, and the ledger returns 409 rather than silently doing the wrong thing. I spent a little care making that fingerprint framing collision proof, so that two different transactions can never accidentally hash the same, but that is a detail for the repo.

11.3 The part I got wrong on the first pass

My first version of the fingerprint glued the fields together with a separator byte, which is the kind of thing that looks fine and passes every test you think to write. Then I pictured a hostile input: what if a field itself contains that separator byte? You could, in principle, craft two different transactions that produce the same glued-together string, collide their fingerprints, and slip a different payment through on a reused key. The fix was to length-prefix every field so the boundaries can never be faked. Nobody would hit this by accident, but “nobody would hit this by accident” is not a sentence you want anywhere near the code that decides whether a payment is a duplicate. I only caught it because I went looking for it, which is the whole argument for building these things yourself.

11.4 The audit log: write it once, never touch it again

The second pattern this week is the audit log. A ledger already keeps immutable postings, so why a separate log? Because the postings tell you *what the balances are*, and the audit log tells you *what happened and who did it*. When an auditor or an incident review asks “show me every action that touched this account, in order, with the full before and after,” you want a single, honest, tamper-evident answer.

Two design decisions make it trustworthy. First, the audit row is written inside the same database transaction as the posting it describes. There is no separate “log this later” step that could fail on its own and leave you with a transaction nobody recorded. If the posting commits, its audit row committed with it. If anything fails, both roll back. They are one fact.

```

1      one database transaction (all of it, or none of it)
2      +-----+
3      | insert transaction + postings          |
4      | insert idempotency key (if one was supplied) |
5      | insert audit_log row: action, actor, snapshot |
6      +-----+
7
8              |
8      commit succeeds
9              |
10     posting and its audit record are now the same
11     indivisible piece of history

```

Second, “append-only” is not just a promise in the application code, it is enforced by the database. A Postgres trigger rejects any attempt to UPDATE or DELETE a row in the audit log. Even a direct `DELETE FROM audit_log` from a psql prompt bounces off it. The only sanctioned exception is the demo seeder that resets `go.sohag.pro` every few hours, and it has to explicitly flip a per-transaction flag to be allowed through. The application’s normal path can never set that flag, so in production the log is genuinely immutable, not immutable-until-someone-writes-the-wrong-migration.

You can read the log back by transaction or by account:

```

1 GET /v1/transactions/{id}/audit    what happened to this transaction
2 GET /v1/accounts/{id}/audit       every audited action touching this
    account

```

The by-account view is paginated with the same keyset cursor as last week's statement endpoint, because "list everything that ever happened to a busy account" is exactly the kind of query that needs a ceiling before it faces a real production account.

11.5 Proving it, not hoping it

The definition of done for this week was a specific, slightly mean test: fire 100 requests with the same idempotency key at the same time, then check the database. Not "it returned 200 a hundred times," but the thing that actually matters: exactly one transaction row exists, exactly one audit row exists, and 99 of the responses were replays of the first. That test runs against a real Postgres in a container, with the race actually happening, not mocked away. Watching it come back green, with the counts landing on exactly one, is a different feeling than reading that unique constraints work. That gap, between knowing a thing and having watched it hold under fire, is the entire reason I am building this in the open.

11.6 My AI companion this week

Same deal as always: I own the decisions, Claude does a lot of the typing. This week the collaboration was interesting because the work was mostly judgment, not code volume. I decided the key had to be inserted inside the posting transaction rather than checked beforehand, and Claude implemented that cleanly and wrote the 100-goroutine hammer test. But the fingerprint collision I described above surfaced during review: a reviewer pass (also Claude, wearing a different hat) flagged the separator-byte weakness, I agreed it was real, and we hardened it before it ever shipped. That is roughly the rhythm I have settled into. I set the invariants and the definition of done, the machine moves fast inside those rails, and the interesting moments are the disagreements and the "wait, what about this input" catches. A payment duplicate check is precisely the code you do not hand over without understanding every line, and I do.

11.7 Where this leaves us

go-ledger now survives retries the way a real payment API has to, and keeps a record of itself that nobody can quietly edit. The API got no bigger in surface area, just harder to misuse, which is the good kind of boring. Next week the ledger grows a second front door: a gRPC layer sitting on the exact same domain services, so REST and gRPC are two protocols over one source of truth rather than two implementations drifting apart.

The repo is open source at github.com/sohag-pro/go-ledger, and the live API is at go.sohag.pro. Go tap Pay twice on something and trust the ledger to have your back.

CHAPTER 12

Week 6 Interview Q&A

Interview-style questions for the Week 6 work: the optional Idempotency-Key header on `POST /v1/transactions`, the exactly-once design that inserts the key row inside the posting's `SERIALIZABLE` transaction, the request fingerprint and 409-on-mismatch contract, the append-only `audit_log` written in the same transaction and enforced by a Postgres trigger, the seeder's gated purge, the keyset-paginated audit reads, and the 100x concurrent test. Every question is answerable from this repo's code, migrations (0006), and tests.

12.1 Idempotency design

12.1.1 1. Walk me through what happens when a client sends `POST /v1/transactions` twice with the same Idempotency-Key.

What a strong answer covers:

- The key is optional: when the header is present the handler builds a `*domain.Idempotency` and passes it into `TransactionService.Post`; when absent it passes `nil` and the post proceeds as before.
- First request: inside one `SERIALIZABLE` transaction it inserts the transaction and postings, inserts the `(tenant_id, idempotency_key)` row, and writes the audit row, then commits.
- Second request with the same key: the key insert hits the primary-key collision, the whole transaction rolls back, the service looks up the stored row, loads the original transaction, and returns it with `Idempotent-Replayed: true`.
- Same key with a different body: returns 409, because the stored fingerprint does not match.

12.1.2 2. Why insert the idempotency key inside the posting transaction instead of checking a table first and then posting?

What a strong answer covers:

- Check-then-act has a race: two concurrent copies of the same request both read “key absent,” both decide to post, and both create a transaction. The bug only appears under concurrency, which is where retries actually happen.
- Making the check and the write one atomic step needs the database as the referee; the unique primary key (`tenant_id`, `idempotency_key`) is that referee.
- Because the key insert shares the transaction with the postings, a losing request’s postings roll back too, so nothing partial survives.
- The result is exactly-once without an application-level lock or a distributed mutex.

12.1.3 3. How does the losing request end up returning the right response after it rolls back?

What a strong answer covers:

- `InsertIdempotencyKey` maps the PK collision (constraint `idempotency_keys_pkey`) to the internal sentinel `domain.ErrDuplicateIdempotencyKey`.
- `Post` catches that specific error (only when `idem != nil`) and calls a replay helper.
- `Postgres` blocks the second insert on the same key until the first transaction resolves, so by the time the loser sees the duplicate error the winner has already committed; the loser’s follow-up read (outside the transaction) is guaranteed to see the committed row.
- `replay` loads the original transaction via the stored `transaction_id`, overwrites the caller’s `*t`, increments `IdempotencyReplays`, and reports `replayed = true`.

12.1.4 4. What is the request fingerprint, and why not just compare the raw request bodies?

What a strong answer covers:

- `Transaction.Fingerprint()` is a SHA-256 over the transaction's semantic content: the currency and each posting's account, signed amount, and description, in order. The transaction id is deliberately excluded.
- Comparing raw bytes would be fragile: whitespace, key order, or an echoed id would make two logically identical requests look different and defeat the replay.
- Comparing the semantic content instead means a genuine retry matches even if serialized slightly differently.
- The stored fingerprint is what distinguishes a true retry (replay) from key reuse with a different body (409 via `ErrIdempotencyConflict`).

12.1.5 5. You length-prefix each field when computing the fingerprint. What attack does that prevent?

What a strong answer covers:

- The first version joined fields with a separator byte; if a field can itself contain that byte, an attacker could shift the boundaries so two different transactions produce the same hash.
- A fingerprint collision on a reused key would let a different payment slip through as a “replay,” which is a correctness and security hole in the duplicate check.
- The fix writes each field's length as a fixed-width prefix before its bytes, so field boundaries cannot be forged regardless of content.
- There is a targeted test with adversarial pairs (embedded NUL, boundary-shift) proving the framed fingerprints stay distinct.

12.2 Data integrity and the audit log

12.2.1 6. Why a separate audit log when postings are already immutable?

What a strong answer covers:

- Postings answer “what are the balances”; the audit log answers “what happened, when, and who did it.”
- For an incident review or an auditor, a single ordered record of actions with

a snapshot is more useful than reconstructing intent from raw postings.

- In v1 it is create-only: one row per posted transaction, `action = transaction.created`, before `NULL`, after a JSON snapshot, actor the tenant id until auth lands.
- It is deliberately not a generic before/after diff framework yet, because nothing mutates in place (YAGNI).

12.2.2 7. How do you guarantee that every posted transaction has an audit row, and no transaction exists without one?

What a strong answer covers:

- The audit insert runs inside the same `RunInTx` closure as the transaction and postings, so they commit together or roll back together.
- There is no “log it afterwards” step that could fail independently and leave a gap.
- On the replay path no audit row is written, because that transaction rolled back and no new posting occurred; the audit log records only real creations.
- The `after` column is `NOT NULL` and the service always marshals a snapshot, so a row can never be half-written.

12.2.3 8. “Append-only” is easy to say. How is it actually enforced?

What a strong answer covers:

- A Postgres trigger on `audit_log` rejects `UPDATE` and `DELETE` (`BEFORE UPDATE OR DELETE ... FOR EACH ROW`), raising an exception; migration 0006 defines it.
- Enforcement is at the database, not just a convention in the app, so even a direct `DELETE FROM audit_log` in psql is refused.
- This mirrors the balance and currency triggers from earlier weeks: defense in depth against a buggy caller or a bad migration.
- The trigger raises with a named constraint so the failure is identifiable rather than an opaque error.

12.2.4 9. The demo seeder wipes the tenant every few hours, but the audit log rejects deletes. How do you reconcile that?

What a strong answer covers:

- The trigger has one gated exception: it allows the mutation only when the transaction-local GUC `audit.allow_purge` is set to on.
- The seeder sets `SET LOCAL audit.allow_purge = 'on'` at the start of its reset transaction, before the deletes; `SET LOCAL` is transaction-scoped, so it cannot leak to other pooled connections.
- The application's normal path never sets the GUC, so in production the log stays immutable; `current_setting(..., true)` returns NULL when unset and the trigger falls through to the exception.
- The seeder's delete order puts `idempotency_keys` and `audit_log` before transactions to respect the foreign keys, and there is a test that reseeds over planted rows to prove the gated purge works.

12.3 Concurrency and correctness

12.3.1 10. Your definition of done was “hammer the same key 100 times, get exactly one transaction.” What exactly does that test assert, and why that shape?

What a strong answer covers:

- It fires 100 concurrent `Post` calls with the same key against a real Postgres (testcontainers), then asserts exactly one transaction row, exactly one audit row, and that 99 responses were replays returning the same id.
- Asserting “returned 200 a hundred times” would pass even if it created 100 transactions; the counts are what prove exactly-once.
- It runs the real race rather than mocking it, so it exercises the unique-constraint referee and the rollback-then-replay path under genuine contention.
- Watching the counts land on one is the difference between believing unique constraints work and having seen it hold under fire.

12.3.2 11. RunInTx retries on serialization failures. Why does a duplicate idempotency key not get retried into an infinite loop?

What a strong answer covers:

- RunInTx only retries when the error is a serialization failure or deadlock (SQLSTATE 40001 / 40P01); it classifies via `isSerializationFailure`.
- A PK collision surfaces as `domain.ErrDuplicateIdempotencyKey`, which is not a `*pgconn.PgError` serialization code, so it is returned immediately, not retried.
- A genuine 40001 on the key insert is still wrapped such that it is detected and retried; on the next attempt it sees the committed key and gets a clean duplicate, which the service replays.
- So the two failure modes are cleanly separated: transient conflicts retry, a real duplicate replays.

12.3.3 12. Could a serialization retry cause a transaction to get two audit rows?

What a strong answer covers:

- No: RunInTx fully rolls back the transaction between attempts and re-runs the whole closure, so a retried attempt starts clean with nothing from the previous attempt persisted.
- The audit insert lives inside that closure, so it is replayed as part of the same all-or-nothing unit; only the attempt that finally commits leaves a row.
- The closure must be safe to run more than once, which it is: it does not mutate external state that survives a rollback.
- This is the same idempotent-closure contract the posting path already relied on from Week 4.

12.4 API and pagination

12.4.1 13. The audit log is queryable by transaction and by account. Why is the by-account endpoint paginated but the by-transaction one is not?

What a strong answer covers:

- A transaction has a fixed, tiny number of audit rows (one, in v1), so it is bounded by nature and a plain list is fine.
- An active account can accumulate audit rows for every transaction that ever touched it, so an unbounded query is a memory and latency risk on a real production account.
- The by-account endpoint reuses the same keyset cursor machinery as the Week 5 statement endpoint (opaque cursor, `next_cursor`, newest-first), rather than an offset.
- This was flagged in review as the one production concern before merge and closed by adding the cursor.

12.4.2 14. How does the Idempotent-Replayed header end up in the OpenAPI spec without hand-editing it?

What a strong answer covers:

- The create handler returns a `CreateTransactionOutput` with a `Replayed` bool field tagged as a response header; huma generates the spec from that typed output, so the header appears automatically.
- The `Idempotency-Key` request header is likewise a tagged input field, and the operation description documents the retry and 409 contract.
- `make_openapi` regenerates the committed `api/openapi.yaml`, and a drift test fails if it is stale, so the published spec cannot diverge from the handlers.
- This keeps the single-source-of-truth property from Week 5 intact.

12.5 Trade-offs

12.5.1 15. Reviewers noted several deferred items: the trigger's gated branch would silently no-op an UPDATE, InsertIdempotencyKey checks the constraint name rather than a generic unique-violation, and there is no PostDuration bucket for replay latency. Why ship with those open?

What a strong answer covers:

- Each was judged safe to defer with a reason: the gate is only ever used for DELETE by the seeder, so a gated UPDATE cannot occur today; the constraint name only appears on a 23505, so the check is behaviorally equivalent to a unique-violation guard.
- The replay-latency gap is observability polish, not correctness; volume is already covered by the IdempotencyReplays and IdempotencyConflicts counters.
- The bar for shipping was the core invariants (exactly-once, atomicity, append-only, tenant isolation), all verified end to end by a whole-branch review, not zero cosmetic findings.
- Naming the deferrals explicitly, rather than silently leaving them, is the point: they are tracked follow-ups, not forgotten corners.

ARCHITECTURE DECISIONS

- 01 ADR-001: Why Double-Entry Accounting
- 02 ADR-002: Money Representation
- 03 ADR-003: Postgres Schema and Repository Layer
- 04 ADR-004: Concurrency Control for Transaction Posting
- 05 ADR-005: Per-Account Currency Integrity
- 06 ADR-006: REST API Design
- 07 ADR-007: Idempotency Keys and the Immutable Audit Log
- 08 ADR-008: Covering Index for Account History Reads

CHAPTER 13

ADR-001: Why Double-Entry Accounting

13.1 Status

Accepted: 2026-06-11

13.2 Context

go-ledger is a payment ledger service: it must record money movements between accounts and answer “what is the balance of account X?” correctly, every time, under concurrent writes.

The naive approach is a single `balances` table with one row per account that gets incremented and decremented in place. This is how many systems start, and it fails in predictable ways:

- **No audit trail.** A mutable balance tells you *what* the balance is, not *how* it got there. Reconstructing history requires separate logging that inevitably drifts from the source of truth.
- **Money appears and disappears.** A single-sided update (“add \$10 to A”) has no structural guarantee that the money came from anywhere. Bugs create or destroy money silently.
- **Race conditions corrupt state.** Read-modify-write on a balance row under concurrency loses updates unless every code path remembers to lock correctly. The schema itself offers no protection.

Double-entry bookkeeping, in use since the 15th century, solves all three structurally:

- Every transaction consists of two or more **postings** (ledger entries).
- Each posting debits one account and credits another.
- The sum of all postings in a transaction must equal zero.

13.3 Decision

go-ledger models all money movement as double-entry transactions:

- Transaction: an atomic, immutable unit of money movement.
- Posting: a single signed entry against one account; a transaction has two or more postings.
- **Invariant:** $\Sigma(\text{postings}) = 0$ for every transaction. Enforced in the domain types (`Transaction.Validate()`), and later at the database level with a CHECK constraint.
- Postings are append-only. Balances are *derived* (sum of postings), never stored as mutable primary state.

13.4 Consequences

13.4.1 Positive

- Money can never be created or destroyed by a write path bug; an unbalanced transaction is rejected before it persists.
- The ledger is its own audit trail: every balance is explainable as a sum of immutable postings.
- Append-only postings sidestep update races entirely; concurrency control reduces to serializing transaction inserts (Week 4).
- The model matches what accountants, auditors, and payment partners expect.

13.4.2 Negative

- Balance reads are aggregations, not single-row lookups. Acceptable for v1; if it becomes hot, a derived (and rebuildable) balance cache can be added; it stays a cache, never the source of truth.
- Slightly more conceptual overhead for API consumers: posting a payment means submitting balanced postings, not “set balance”.

13.5 Alternatives considered

- **Mutable balance table:** rejected. No audit trail, no structural integrity guarantee (see Context).
- **Event sourcing with a full event store:** rejected for v1. Double-entry postings already give us the append-only history we need without the operational complexity of projections and replay. Revisit only if product needs demand it (see scope discipline in the build plan).

CHAPTER 14

ADR-002: Money Representation

14.1 Status

Accepted: 2026-06-15

14.2 Context

The ledger moves money between accounts and must answer balance queries exactly. Floating point is unfit for money: $0.1 + 0.2 \neq 0.3$ in IEEE-754, and rounding error accumulates across millions of postings. We need an exact, fast representation that maps cleanly to Postgres in Week 3.

Three candidates:

- **int64 minor units**: store the amount as a signed integer count of the currency's smallest unit (cents for USD). Exact, no allocation, maps to BIGINT. Must guard against overflow on arithmetic.
- **Arbitrary-precision decimal** (for example shopspring/decimal): exact and overflow-free, but every operation allocates a big.Int, and it maps to NUMERIC. More machinery than a single-currency v1 needs.
- **Custom decimal struct** (int64 coefficient + scale): full control, but we own the arithmetic, rounding, and a pile of tests for no v1 benefit.

14.3 Decision

go-ledger represents money as int64 minor units in a Money value type that also carries its Currency. v1 is single-currency (USD), per the build plan's scope discipline, so there is no per-currency scaling to track yet.

- Money is an immutable value type. Operations return new Money values.
- Arithmetic (Add, Sub, Neg) is overflow-guarded and returns an error on overflow rather than panicking or wrapping.

- Cross-currency arithmetic returns `ErrCurrencyMismatch`. The check exists now so multi-currency (a future, out-of-scope item) cannot silently corrupt sums.
- Maps to `BIGINT` in Postgres (Week 3) with no conversion.

14.4 Consequences

14.4.1 Positive

- Exact arithmetic with zero allocation on the hot path.
- Clean, lossless mapping to `BIGINT`; no `NUMERIC` parsing.
- Overflow is a typed, testable error, not undefined behavior.

14.4.2 Negative

- `int64` caps the representable amount near $9.2e16$ minor units. Far beyond any realistic single-account balance, but arithmetic must still check, hence the overflow guard.
- Adding a second currency later means introducing per-currency exponents. Acceptable: multi-currency is explicitly out of scope for v1.

14.5 Alternatives considered

- **float64**: rejected. Not exact; unacceptable for money.
- **shopspring/decimal**: rejected for v1. Allocation and `NUMERIC` overhead with no benefit while we are single-currency. Revisit if multi-currency lands.
- **Custom decimal struct**: rejected. Owning rounding and arithmetic is cost without v1 value.

CHAPTER 15

ADR-003: Postgres Schema and Repository Layer

15.1 Status

Accepted: 2026-06-29

15.2 Context

Week 3 puts the domain model (ADR-001, ADR-002) onto disk. We need a schema for accounts, transactions, and postings, a repository layer over it, and an integration test that runs the full happy path against a real Postgres. Several choices had no obvious default and would be costly to reverse once data exists, so each is recorded here.

The hardest question is how balances are stored. Two shapes:

- **Mutable balance column:** each account row carries a balance updated in place on every posting. Reads are a single column lookup.
- **Append-only postings, derived balances:** postings are immutable rows; balance is $\text{SUM}(\text{amount})$ over an account. Nothing is updated in place.

A mutable balance is faster to read but reintroduces exactly the bug double-entry exists to prevent: the stored number and the sum of postings can drift, and once they drift there is no way to tell which is right. It also turns every posting into a read-modify-write that must be serialized.

15.3 Decision

Postings are append-only and balances are derived. This carries the core invariant from ADR-001 into the database. The schema, with the reasoning for each non-obvious choice:

- **Three tables now, not five.** The build plan lists five tables, but `idempotency_keys` (Week 6) and `audit_log` (Week 6) are designed and built in the weeks that use them, when their columns are actually known. `0001_initial` creates only accounts, transactions, and postings. Migrations are cheap; guessing a schema before its code exists is not.
- **Multi-tenant from day one.** Every table carries a `tenant_id`. There is no auth or tenant resolution yet (that comes later); the column and the foreign key discipline exist now because retrofitting tenancy onto a populated ledger is a migration no one wants to write. Repository methods are all scoped by tenant.
- **UUIDv7 primary keys, generated in the application.** Keys are `uuid` columns; the app generates a UUIDv7 before insert. v7 is time-ordered, so it keeps index locality close to a serial key while staying globally unique and decoupled from the database. Generating in Go (not `gen_random_uuid()`) means the caller knows the id without a round-trip and tests are deterministic.
- **Composite foreign keys enforce tenant integrity.** `accounts` and `transactions` carry a `UNIQUE (tenant_id, id)`. `postings` references both via `(tenant_id, account_id)` and `(tenant_id, transaction_id)`. A posting cannot point at an account or transaction in a different tenant: the database rejects it, not just application code.
- **Currency lives on the transaction, not the posting.** The domain already enforces that all postings of a transaction share one currency (`Transaction.Validate`). Storing it once on the transaction keeps a posting to a single signed `bigint` amount, and each posting's `Money` is reconstructed from the transaction's currency on read. Accounts also carry their own currency.
- **Account type is text with a CHECK, not a Postgres enum.** The five types are constrained by `CHECK (type IN (...))`. Readable in plain SQL, and adding or changing a type is an ordinary migration rather than the awkward `ALTER TYPE` dance an enum forces.
- **No balance CHECK constraint yet.** The sum-to-zero invariant is enforced in the domain (`Transaction.Validate`) for Week 3. The database-level `CHECK` lands in Week 4 alongside the transaction-posting service and its concurrency handling, where it belongs.

The repository follows a port-and-adapter split: `domain.Repository` is the port (the domain owns the contract); `internal/postgres` is the adapter, built on `pgx` and `sqlc`-generated queries. `CreateTransaction` validates the double-entry invariant, then writes the transaction and all its postings in one database transaction, so a half-written transaction can never exist. Tooling is `goose` for migrations, `sqlc` for type-safe queries, and `testcontainers-go` for the integration

test (all locked in the build plan).

15.4 Consequences

15.4.1 Positive

- The stored ledger cannot disagree with itself: there is no second copy of the balance to drift from the postings.
- Tenant isolation is enforced by the database, not just by careful queries.
- The domain stays free of storage and uuid types; the port is a plain Go interface, so the service layer (Week 4) can be tested against a fake.
- UUIDv7 keeps write-side index locality without coupling ids to the database.

15.4.2 Negative

- Balance reads are an aggregate (SUM) rather than a column read. Fine at v1 scale; if it ever bites, the answer is a materialized rollup that is rebuilt from postings, never a mutable primary balance.
- The integration test needs a Docker daemon. It skips (does not fail) when none is reachable, so `local make test` stays green; CI runs Docker and exercises the real path.

15.5 Alternatives considered

- **Mutable balance column:** rejected. Reintroduces drift between stored balance and posting history, the exact failure double-entry prevents, and serializes every posting behind a row update.
- **All five tables in 0001:** rejected. Designs `idempotency_keys` and `audit_log` before the code that uses them; they get their own migrations in Week 6.
- **Single-tenant schema, add tenancy later:** rejected. Adding `tenant_id` and backfilling foreign keys on a live ledger is far more painful than carrying the column from the start.
- **DB-generated UUIDv4 (`gen_random_uuid()`):** rejected. Random keys hurt index locality and force a read-back to learn the generated id.

- **Postgres enum for account type:** rejected. ALTER TYPE is more friction than a text column with a CHECK, for no real gain.

CHAPTER 16

ADR-004: Concurrency Control for Transaction Posting

16.1 Status

Accepted: 2026-06-29

16.2 Context

Posting a transaction writes a transaction row and two or more postings that must sum to zero, and it must do so correctly when many requests post to the same accounts at once. Two requests racing on the same account cannot be allowed to produce a ledger that does not balance, lose a posting, or read a stale balance.

The schema (ADR-003) already makes postings append-only and balances derived, so there is no mutable balance row to corrupt. What remains is making each posting atomic and keeping the double-entry invariant intact under concurrency.

There are two classic strategies:

- **Pessimistic: lock the rows.** Run at READ COMMITTED and SELECT ... FOR UPDATE the involved account rows, in a fixed order to avoid deadlock, before writing. Conflicting transactions block rather than fail, so there is nothing to retry. Correctness rides on remembering to lock the right rows, in the right order, every time.
- **Optimistic: let the database detect conflicts.** Run at SERIALIZABLE. PostgreSQL's Serializable Snapshot Isolation (SSI) tracks read/write dependencies and aborts a transaction with SQLSTATE 40001 if committing it would break serial-equivalent ordering. No explicit locks; conflicts surface as errors the application retries.

16.3 Decision

go-ledger uses **SERIALIZABLE with automatic retry**. Correctness comes from the database guaranteeing serial-equivalent execution, not from the application remembering to take the right locks. A buggy or future code path cannot silently skip a lock it never knew to take.

The pieces:

- **Service owns the unit of work.** `ledger.TransactionService.Post` validates the transaction, then runs the write through the repository's `RunInTx`, which begins a **SERIALIZABLE** transaction, runs the work, and commits.
- **The adapter owns isolation and retry.** `RunInTx` (in `internal/postgres`) begins each attempt at `pgx.Serializable`. A serialization failure (40001) or deadlock (40P01) can surface either inside the work or, commonly, only at `COMMIT`, so both are watched and the whole unit of work is replayed. Retries are bounded (25 attempts) with exponential backoff and full jitter; the jitter is what stops a crowd of conflicting transactions from retrying in lockstep and colliding again.
- **The invariant is also enforced in the database.** A `DEFERRABLE INITIALLY DEFERRED` constraint trigger (`assert_txn_balanced`, migration 0002) runs at `COMMIT` and rejects any transaction whose postings do not sum to zero. This is defense in depth: the domain checks the invariant first (`Transaction.Validate`) for a fast, clean error, but the trigger guarantees it holds even against a bad migration, a direct `psql` write, or a second service.
- **The trigger's read is indexed.** The trigger reads postings by `transaction_id`. Without an index that read is a scan, and under SSI a scan takes broad predicate locks that turn into a storm of false-positive serialization conflicts. Adding `postings_transaction_idx` was the single change that took the stress test from roughly 1 in 6 posts failing to zero. This is the non-obvious lesson of the week: under **SERIALIZABLE**, an unindexed read is not just slow, it manufactures conflicts.

`SELECT FOR UPDATE` is not used. The two approaches were not combined: belt and suspenders here would mean taking explicit locks and running **SERIALIZABLE** and retrying, which is more moving parts and redundant guarantees for no gain.

16.4 Observability

Posting is instrumented with two Prometheus collectors: a histogram `transaction_post_duration_seconds`, labeled by outcome, for latency, and a counter `transaction_post_serialization_retries_total` so a climbing retry rate makes write contention visible. These sit alongside the standard Go runtime and process collectors.

The scrape endpoint is served on a separate server bound to loopback (127.0.0.1:9090 by default), not on the public API port. A metrics endpoint leaks transaction volumes, latencies, and retry rates, so it is reached over the host's private network or an SSH tunnel, never the public interface. The public liveness check stays `GET /healthz`.

16.5 Measured baselines

Stress test: 100 goroutines posting 10,000 balanced transactions against a shared pool of 100 accounts. Result: all 10,000 commit, zero failures, and the sum of every account balance is exactly zero.

One honest caveat about the concurrency level: the real ceiling on how many posting transactions run at the database at once is the connection pool's `MaxConns`, not the goroutine count. The test sets `MaxConns` to 25 explicitly and logs it, so "100 goroutines" means up to 25 transactions truly concurrent at Postgres with the rest queued on pool acquisition. That is still meaningful contention on 100 shared accounts, and it is the number worth quoting rather than the goroutine count.

Latency on the local test machine (Postgres 16 in a colima VM): p50 about 35 ms, p99 about 68 ms. These are above the original `p50 < 10 ms / p99 < 50 ms` target. The dominant cost is WAL fsync on the VM's virtualized disk: every commit waits for a durable write, and a Lima VM's disk is slower than bare metal. The numbers are reported as measured rather than tuned (for example by disabling `synchronous_commit`), since durability is the point of a ledger. On bare-metal or a real server disk the same code should land much closer to the target. Latency is logged by the test, not asserted, so the suite does not flake on machine speed.

16.6 Consequences

16.6.1 Positive

- Correctness does not depend on the application taking the right locks; the database guarantees serial-equivalent execution.
- The double-entry invariant is enforced in two independent places: the domain and a deferred database trigger.
- Retry with jittered backoff absorbs contention transparently; under the test's sustained load every transaction eventually commits.
- `RunInTx` is a general unit-of-work seam, so Week 6 can write the audit-log entry inside the same transaction as the posting it records.

16.6.2 Negative

- Under contention some transactions do extra work (retries), and pathological contention could still exhaust the retry budget and surface an error to the caller. That is the optimistic trade: conflicts cost retries, not blocked connections.
- SSI is sensitive to read patterns. Indexes that look optional for correctness (like the one on `transaction_id`) are load-bearing for concurrency. Future reads inside a posting transaction must be considered for their SSI footprint.
- `fn` passed to `RunInTx` must be safe to run more than once, since it can be replayed.

16.7 Alternatives considered

- **READ COMMITTED + SELECT FOR UPDATE**: rejected. Pushes correctness onto the application to lock the right rows in the right order every time; a future code path that forgets reintroduces the race. Pessimistic locking also serializes access to hot accounts by blocking, rather than letting non-conflicting work proceed.
- **SERIALIZABLE + SELECT FOR UPDATE + retry**: rejected. Redundant guarantees and more moving parts than **SERIALIZABLE** alone needs.
- **Mutable balance with row locks**: rejected back in ADR-003; it reintroduces a second copy of the truth that can drift.

- **A non-deferred row CHECK:** impossible; the sum-to-zero invariant spans many posting rows and cannot be expressed in a single-row CHECK, hence the deferred constraint trigger.

CHAPTER 17

ADR-005: Per-Account Currency Integrity

17.1 Status

Accepted: 2026-06-29

17.2 Context

Currency lives in two places (ADR-003): every account has a currency, and every transaction has a currency (the single currency shared by all its postings). Postings themselves carry only a signed amount, not a currency, because the domain already guarantees all postings of a transaction share one currency.

That leaves a gap. Nothing tied a transaction's currency to the currency of each account it posts into. A USD transaction could post into an account whose currency is EUR, and the database would accept it. Today the ledger is single-currency (USD), so the gap is invisible, but it is exactly the kind of latent integrity hole that turns into a real one the moment multi-currency, a second writer, or a buggy import appears. A reviewer would not trust a ledger that lets a posting land in an account of a different currency.

The domain cannot close this on its own: `Transaction.Validate` sees the postings and their shared currency, but it does not know each referenced account's currency, which lives in the database.

17.3 Decision

Enforce it in the database. Migration 0003 adds an immediate `AFTER INSERT` trigger on postings (`assert_posting_currency`) that, for each inserted posting, looks up its account's currency and its transaction's currency and rejects the row if they differ.

- It is immediate, not deferred: the account and transaction both already

exist when a posting is inserted (the foreign keys require it), so there is nothing to wait for.

- The lookups are equality on primary keys, so under `SERIALIZABLE` they take only narrow predicate locks on rows that posting transactions read but never write. The stress test confirmed this adds no measurable serialization contention.

This is defense in depth, consistent with how the balance invariant is handled (ADR-004): the domain keeps postings in one currency, and the database guarantees that currency matches each account, even against a direct write.

17.4 Consequences

17.4.1 Positive

- A posting can never move money in a currency the account does not hold. The guarantee holds regardless of what client or code path wrote the row.
- The schema is ready for multi-currency: when it lands, this invariant is already enforced rather than retrofitted onto live data.

17.4.2 Negative

- A small per-posting read cost at insert time (two primary-key lookups). Negligible in practice and confirmed not to add serialization conflicts.
- The currency rule now lives in two places: the domain (postings share one currency) and the database (that currency matches the account). That redundancy is the point, but it is two places to keep in mind.

17.5 Alternatives considered

- **Denormalize currency onto postings and use a composite foreign key** `postings(account_id, currency)-> accounts(id, currency)`: rejected. It would put currency back on the posting row, undoing the ADR-003 decision to keep a posting to a single signed amount, for no extra safety beyond what the trigger gives.

- **Enforce only in application code** (check each account's currency in `CreateTransaction`): rejected. That is the very thing we want the database to guarantee; an application-only check is one forgotten code path away from a cross-currency posting.
- **Do nothing until multi-currency arrives**: rejected. Adding the constraint to a populated, multi-currency ledger later is far riskier than enforcing it now while the rule is trivially true.

CHAPTER 18

ADR-006: REST API Design

18.1 Status

Accepted: 2026-06-29

18.2 Context

Through Week 4 the ledger was exercised only by tests; the service had no network surface. Week 5 puts a REST API in front of it: create accounts, post transactions, and read accounts, balances, statements, and transactions. The router is chi (a locked decision), and the project already generates its OpenAPI spec from the handlers with huma, with a drift test and a self-hosted playground.

Several choices had no obvious default.

18.3 Decision

18.3.1 huma operations on chi

The endpoints are huma operations running on a chi router via the humachi adapter (swapped from humago). Each endpoint is a typed operation: huma derives request validation from struct tags, renders errors as RFC 7807 application/problem+json, and generates the OpenAPI spec, all from the same handler. chi owns routing and the middleware chain (request id, recovery, structured request logging).

This was chosen over plain chi handlers plus go-playground/validator plus hand-written problem+json (the build plan's literal wording). huma already solves validation, error formatting, and spec generation as one unit, and the existing drift test and playground depend on the spec being generated

from handlers. The plan predates the huma decision; huma's validation supersedes the separate validator cleanly. The cost is using huma's validation tags (`minLength`, `enum`, `pattern`) rather than go-playground's, which is minor.

18.3.2 Money as integer minor units

Amounts cross the wire as a signed integer count of minor units plus a currency code (`{"amount": 10000, "currency": "USD"}`), mapping 1:1 to the internal `int64`. No floating point and no decimal parsing at the boundary. This is what most payment APIs (for example Stripe) do, and it keeps the API as exact as the ledger.

18.3.3 Single default tenant for now

The schema is multi-tenant, but there is no auth yet. Rather than expose tenancy in URLs or trust a client header, every request acts as a single default tenant (`DEFAULT_TENANT_ID`, with a fixed fallback), injected by the API layer into each service call. When auth lands it will populate the same tenant value from a token instead, without changing any resource URL.

18.3.4 Account statement with running balance and keyset paging

Beyond the planned endpoints, `GET /v1/accounts/{id}/statement` lists the account's postings, newest first, each with the account's running balance as of that posting (a window `SUM` over the account's history), and pages with keyset (cursor) pagination on (`created_at`, `id`). Keyset paging is stable under concurrent inserts in a way offset paging is not.

Because balances are derived, never stored (ADR-003), the running-balance window scans the account's full posting history per page (one index range on (`tenant_id`, `account_id`)). That is $O(\text{history})$ per page, fine at v1 scale; the future optimization, if it ever bites, is a materialized running balance rebuilt from postings, never a mutable primary balance.

18.3.5 Per-posting description

Each posting carries an optional free-text description (narration), bounded at 256 characters in both the domain and a database CHECK. It appears on every posting in requests and responses, including statement entries.

18.3.6 Server now owns its database, with startup migrations

`cmd/server` reads `DATABASE_URL` (required, fail-fast: a ledger API with no database must not serve), opens the tuned pool, and applies the embedded goose migrations on startup before serving. On a single instance this is the simplest correct option: the binary that needs a column also creates it, atomically with the deploy. If the project ever runs more than one instance, the same `goose.Up` moves into a dedicated CI step to avoid a migration race.

Production Postgres runs natively on the VPS (a dedicated database and user, listening on localhost, backed up with `pg_dump`), consistent with the native-binary deploy. This is provisioned out of band before the deploy; see the ops runbook.

18.3.7 Error mapping

Domain errors map to status codes in one helper: not-found to 404, duplicate id to 409, an exhausted serialization conflict to 503, validation and invariant failures to 422, and anything unrecognized to a 500 that does not leak internals.

18.4 Consequences

18.4.1 Positive

- One source of truth for handlers, validation, errors, and the spec; the published docs cannot drift from the running API.
- The API is as exact as the ledger (integer minor units), and statements give a real running balance with stable pagination.
- The transport stays thin: services hold the logic, handlers translate.

- Auth can be added later by changing only how the tenant is resolved.

18.4.2 Negative

- huma owns request decode/encode, so a very custom response shape means working within its model.
- Statement pages cost a scan of the account's posting history while balances stay derived. Accepted for v1; a rollup is the escape hatch.
- The server now hard-depends on Postgres, so the deploy requires the database to be provisioned and DATABASE_URL set first.

18.5 Alternatives considered

- **chi + go-playground/validator + hand-written problem+json**: rejected. Reintroduces three concerns huma already covers and risks spec drift.
- **Spec-first with oapi-codegen**: rejected. Drops the existing huma investment (playground, drift test, generator) and makes the spec a hand-maintained artifact, heavy for a solo build.
- **Decimal-string money in JSON**: rejected. Adds parsing and per-currency scale validation at the boundary for no gain over exact integer minor units.
- **Tenant in the URL or a trusted header now**: rejected. Bakes tenancy into the resource model or trusts an unauthenticated header; the default-tenant seam is cleaner to evolve into real auth.
- **Migrations as a separate step now**: deferred. Correct for multi-instance, but premature for a single VPS; startup migrations are simpler today.

CHAPTER 19

ADR-007: Idempotency Keys and the Immutable Audit Log

19.1 Status

Accepted: 2026-07-02

19.2 Context

Through Week 5 the ledger had a REST API but no protection against retries. A `POST /v1/transactions` that times out may have already committed: the request reached the server and posted, and only the response was lost. A correct client retries, and without protection that retry posts a second transaction, moving money twice. Week 6 adds an idempotency contract so a retry is safe, and an audit log so every posting leaves a tamper-evident record of what happened.

Both tables (`idempotency_keys`, `audit_log`) were designed in ADR-003 and left for this week; `RunInTx` was called out in ADR-004 as the seam the audit write would use. This ADR records the decisions those earlier ADRs deferred.

Several choices had no obvious default.

19.3 Decision

19.3.1 The idempotency key row is inserted inside the posting transaction

The client sends an optional Idempotency-Key header. When present, the key row is inserted into `idempotency_keys` inside the same `SERIALIZABLE` transaction (`RunInTx`) that inserts the transaction, its postings, and the audit row. The primary key (`tenant_id`, `idempotency_key`) is the exactly-once mutex: the

first request to commit the key wins, and any concurrent duplicate hits the unique violation, rolls back its whole transaction (so its optimistically written postings vanish too), and replays.

This was chosen over the obvious check-then-act approach (look the key up, and if absent, post). Check-then-act has a race: two concurrent copies of the same request both read “key absent,” both post, and both create a transaction. The bug only appears under concurrency, which is exactly when retries happen. Making the check and the write one atomic step requires the database as the referee; the unique primary key is that referee, so no application lock or separate idempotency service is needed.

`InsertIdempotencyKey` maps the PK collision (constraint `idempotency_keys_pkey`) to an internal `ErrDuplicateIdempotencyKey`. `TransactionService.Post` catches it (only when a key was supplied), reads back the committed row, loads the original transaction, and returns it with `replayed = true`, which the API surfaces as the `Idempotent-Replayed` response header. Because Postgres blocks the second insert until the first transaction resolves, the loser’s follow-up read always sees the committed winner; there is no visibility gap.

19.3.2 A request fingerprint distinguishes a retry from key reuse

The stored key row holds a fingerprint of the original request: `Transaction.Fingerprint()`, a SHA-256 over the currency and each posting’s account, signed amount, and description, in order, with the transaction id excluded. On a repeat, a matching fingerprint is a genuine retry and gets the replay; a different fingerprint means the client reused a key for a different payment, and the service returns `ErrIdempotencyConflict`, mapped to HTTP 409.

The fingerprint length-prefixes each field rather than joining with a separator byte. A separator scheme is forgeable when a field can contain that byte: two different transactions could be crafted to collide, and a collision on a reused key would let a different payment through as a “replay.” Fixed-width length prefixes make field boundaries impossible to fake regardless of content.

19.3.3 The idempotency key is optional, not required

An absent header posts normally, exactly as before Week 6. This keeps the change backward compatible (the console and seeder callers are unaffected) and matches common payment APIs where the key is recommended but not mandatory. A keyless post is still audited.

19.3.4 The audit log is written in the same transaction and is create-only

Each posted transaction writes one `audit_log` row inside the same `RunInTx` closure as the postings: `action = transaction.created`, before `NULL`, after a JSON snapshot of the transaction, actor the tenant id (until auth lands). Because the audit insert shares the transaction, a posting and its audit record commit together or roll back together; there is no separate “log it later” step that could fail on its own and leave a transaction nobody recorded. On the replay path the transaction rolled back and no new posting happened, so no audit row is written: the log records only real creations.

It is deliberately create-only. Postings are append-only, so there is no in-place update to diff; a generic before/after framework would be speculative (before stays `NULL` for now).

19.3.5 Append-only is enforced by a database trigger, with one gated exception

A trigger on `audit_log` rejects `UPDATE` and `DELETE` (`BEFORE UPDATE OR DELETE ... FOR EACH ROW`), so the log is immutable even against a direct `DELETE FROM audit_log` in `psql`, not just by application convention. This is the same defense-in-depth pattern as the balance and currency triggers (ADR-004, ADR-005).

The one sanctioned exception is the demo seeder, which resets the tenant every few hours and must clear the log. The trigger allows the mutation only when the transaction-local GUC `audit.allow_purge` is set to `on`; the seeder sets `SET LOCAL audit.allow_purge = 'on'` at the start of its reset transaction. `SET LOCAL` is transaction-scoped, so it cannot leak to other pooled connections, and the application’s normal path never sets it, so production stays immutable.

19.3.6 Audit reads: by transaction unpaginated, by account keyset-paginated

GET /v1/transactions/{id}/audit returns a transaction's audit rows unpaginated, because a transaction has a fixed, tiny number of them. GET /v1/accounts/{id}/audit reuses the Week 5 keyset cursor on (created_at, id), because an active account can accumulate a row for every transaction that ever touched it, and an unbounded query is a memory and latency risk on a real account. The by-account query joins audit_log to the account's postings and is tenant scoped on both the outer table and the subquery.

19.4 Consequences

19.4.1 Positive

- Retrying a payment is safe: 100 concurrent requests with the same key create exactly one transaction and one audit row, verified by an integration test that runs the real race against Postgres.
- Exactly-once needs no distributed lock or idempotency service; a unique constraint inside the existing transaction does it.
- Every posting leaves an immutable, transactionally-consistent audit record that the database itself refuses to alter.
- The change adds no API surface that can be misused: the key is optional and the audit reads are bounded.

19.4.2 Negative

- The idempotency key table grows without bound in v1 (no TTL or expiry); the demo seeder clears it on reset, and real retention is a later concern.
- The audit after snapshot duplicates data already in the postings; accepted, as the point of an audit log is a self-contained record.
- The immutability trigger's gated branch currently permits any mutation (not just DELETE) when the GUC is set; only the seeder's DELETE uses it today, so a gated UPDATE would silently no-op. Tracked as a follow-up (fail closed on UPDATE).
- Replay and conflict paths are counted (IdempotencyReplays, IdempotencyConflicts)

) but not timed in the `PostDuration` histogram.

19.5 Alternatives considered

- **Check-then-act idempotency (look up, then post):** rejected. Races under concurrent retries and creates duplicate transactions exactly when it matters.
- **A separate idempotency middleware with its own table and short transaction:** rejected. The key insert would not share the posting's transaction, reopening a window where the posting commits but the key does not (or two requests both proceed); keeping the mutex inside the posting transaction is simpler and strictly correct.
- **Required idempotency key:** rejected for v1. A breaking change to the existing contract and to the console and seeder callers, for a guarantee clients can opt into per request.
- **Fingerprint by hashing the raw request body:** rejected. Whitespace, key order, or an echoed id would make identical payments look different and break the replay; hashing the semantic content is stable.
- **Separator-byte fingerprint framing:** rejected. Forgeable when a field can contain the separator, allowing a collision on a reused key; length-prefixing is collision-safe.
- **Audit log immutability by convention only:** rejected. A trigger enforces it at the database, consistent with the ledger's other invariants, so a buggy caller or a bad migration cannot rewrite history.
- **Generic before/after audit framework now:** deferred. Nothing mutates in place, so create-only with a NULL before is enough; a diff model is speculative.
- **Unpaginated by-account audit read:** rejected. Fine for the demo, but an unbounded query is a production memory and latency risk; keyset paging matches the statement endpoint.

CHAPTER 20

ADR-008: Covering Index for Account History Reads

20.1 Status

Accepted: 2026-07-02

20.2 Context

ADR-006 documents that the account statement is $O(\text{history})$ per page: because balances are derived, never stored (ADR-003), the running-balance window scans the account's full posting history within a single index range on $(\text{tenant_id}, \text{account_id})$, the two-column `postings_tenant_account_idx` from `0001_initial.sql`. ADR-007 added a second reader on the same shape: `GET /v1/accounts/{id}/audit` keyset-pages $(\text{created_at}, \text{id})$ through a join from `audit_log` to the account's postings.

Both queries filter on $(\text{tenant_id}, \text{account_id})$ and then need their rows ordered by $(\text{created_at}, \text{id})$: the statement to return newest-first pages with a correct running balance, the audit-by-account read to keyset-page in the same order. The two-column index covers the filter but not the order, so Postgres has to sort every page's matched rows after the index range scan. On a busy account this sort runs on every single page request, competing for the one CPU core the production VPS has.

A coverage/capacity audit flagged this as the read-side gap most likely to show up first as the demo tenant's ~ 285 seeded transactions grow, and it is cheap to close without touching query text or the derived-balance model.

20.3 Decision

Migration 0007 replaces `postings_tenant_account_idx (tenant_id, account_id)` with `postings_tenant_account_created_idx (tenant_id, account_id, created_at, id)`.

The composite's leading two columns, `(tenant_id, account_id)`, still serve the balance `SUM` and any other equality-only lookup, exactly as the old index did. The trailing `(created_at, id)` means an index range scan on the tenant/account prefix already returns rows in the exact order the statement and audit-by-account queries need, so Postgres can walk the index directly into a page of results instead of scanning then sorting. No `sqlc` query text changes: the SQL was already selecting and ordering by these columns, it was just the index that did not match.

The old two-column index is dropped in the same migration. Every query that could use it can use the composite's leading prefix instead, so keeping both would only add write-side index maintenance for no read benefit.

This does not change what is derived versus stored. Balances are still computed by summing postings at read time; the index removes the per-page sort, it does not cache or materialize a balance. The $O(\text{history})$ scan cost ADR-006 already accepted is unchanged: this is a narrower fix for the sort that sat on top of it.

20.4 Consequences

20.4.1 Positive

- Statement and audit-by-account pages no longer sort after the index scan; the index range scan already returns rows in the order the query needs.
- No application or query-text change: `internal/postgres` and its `sqlc` queries are untouched, only the schema.
- One index does the work of two: the composite serves every read the old two-column index served, plus the ordered scans, at one index's write cost instead of two.

20.4.2 Negative

- The composite index is wider than the one it replaces (two extra columns per entry), so it costs slightly more to maintain per insert. On the production box's single core and 1GB of memory this is a small but real addition to write-side work; accepted, since posting throughput is not the bottleneck the audit flagged.
- Still $O(\text{history})$ per page in the sense that the scanned range grows with an account's full posting count, even though it is no longer sorted. A genuinely unbounded account still needs the materialized-rollup escape hatch ADR-006 already named, not another index.

20.5 Alternatives considered

- **Keep both indexes:** rejected. The two-column index becomes fully redundant once the composite exists (same leading prefix), so this would just pay index maintenance twice for the same set of queries.
- **Materialized running balance, rebuilt from postings:** deferred. This is the actual escape hatch ADR-003 and ADR-006 already named for when $O(\text{history})$ itself becomes the problem, not just the sort on top of it. Out of scope for this change: it would introduce a second, cached number that has to reconcile with the postings, which is the exact drift ADR-003 rejected a mutable balance column to avoid. Revisit only if index-scan cost itself, not the sort, becomes the bottleneck.
- **Do nothing:** rejected. Leaves an unbounded per-page sort competing for the single core on every statement and audit-by-account request, and the fix is a single, low-risk, additive migration.

≡ `go-ledger`



Built in public, one week at a time.

Read, clone, follow along

> github.com/sohag-pro/go-ledger

> go.sohag.pro

> notes.sohag.pro/series/go-ledger